Széchenyi István Egyetem
Gépészmérnöki, Informatikai és Villamosmérnöki Kar
Informatika Tanszék

SZAKDOLGOZAT

Balogh Attila

Mérnök Informatikus BSc szak

2025

Balogh Attila, 2025

SZÉCHENYI EGYETEM
UNIVERSITY OF GYŐR

INFORMATIKA TANSZÉK
DEPARTMENT OF INFORMATICS

SZAKDOLGOZAT

Legends of the Hero: A Platformer Adventure

Balogh Attila

Mérnök Informatikus BSc szak

2025

1/b. melléklet: Feladat-kiíró lap diplomamunkához

Feladat-kiíró lap diplomamunkához

Hallgató adatai
Név: Balogh Attila *Neptun-kód: HRFUT5*
Szak: Mérnökinformatikus - IVIN_BMI e-mail cím: battis040414@gmail.com
Tagozat: nappali

A diplomamunka adatai
Kezdő tanév és félév (A kurzus felvételének féléve): 2024/25/2
Nyelv: magyar
Típus: nyilvános

Legends of the Hero: A Platformer Adventure

Feladatok részletezése:

A szakdolgozatom célja, hogy bemutassam egy 2D-s platformer játék tervezési és fejlesztési folyamatát a Unity játékmotor használatával. A játékban egy hős szerepét öltöm magamra, aki különböző kihívásokkal és akadályokkal szembesül egy pályán, miközben a végcélt, mint például egy hercegnő kiszabadítása, tűzi ki maga elé. A dolgozatban a tervezés, a játékmenet kialakítása, az AI és az interakciók részletes bemutatása mellett célom, hogy megjelenítsem a platformer játékok készítésében alkalmazott technikai és kreatív megoldásokat.

A játék fő karaktere egy hős, aki egy változatos pályán halad végig, miközben egyre nehezedő akadályokkal, ellenfelekkel és megmérettetésekkel kell szembenéznie. Az egyes pályarészek egyre nagyobb kihívást jelentenek a játékos számára, az ellenfelek és a környezeti akadályok fokozatosan nehezednek. A végén egy boss fight is szerepel, amely nemcsak a harci képességeket, hanem az AI megoldások alkalmazását is hangsúlyozza, hogy a játék még izgalmasabb és dinamikusabb legyen. A projekt során különös figyelmet szentelek a játékmenet élvezhetőségére, az AI ellenségek és a boss mesterséges intelligenciájának megtervezésére. A játéknak folyamatosan egyensúlyban kell tartania a kihívások mértékét, így a nehézségi szintek dinamikus állítására és az ellenségek viselkedésére is szükség lesz. Az AI megoldások lehetővé teszik, hogy a boss és más ellenfelek a játék során rugalmasan alkalmazkodjanak a játékos mozgásaihoz, így egy-egy nagyobb csata vagy összecsapás igazán izgalmas élményt nyújtson.

A projekt főbb részfeladatai:

- A hős mozgásának, interakcióinak és alapvető játékmeneti mechanikáinak kialakítása.

- Különböző pályarészek és kihívások tervezése a fokozódó nehézség érdekében.
- A pályaelemek, akadályok és ellenségek dinamikus elrendezése a játékmenet szórakoztató és kihívásokkal teli fenntartása érdekében.
- A boss fight mesterséges intelligenciájának kidolgozása, amely kiemelkedő kihívást és dinamikus játékmenetet biztosít.
- A nehézségi szintek megfelelő balanszírozása a szórakoztató és motiváló élmény érdekében.
- Az animációk és karakterek közötti interakciók megtervezése, különös figyelmet fordítva az akadályokkal való érintkezésekre, valamint az ellenségekkel való harcra.
- A játék alapos tesztelése különböző szempontok szerint, mint például játékmenet, AI viselkedés, és hibák keresése.

A dolgozat során bemutatott játékmenet, tervezési folyamatok és technikai megoldások célja, hogy egy szórakoztató, kihívásokkal teli játékélményt nyújtson a felhasználóknak. A játék fejlesztése nemcsak a játékmenet folyamatos finomhangolásáról szól, hanem arról is, hogyan lehet hatékonyan alkalmazni a mesterséges intelligenciát egy dinamikus és interaktív élmény megteremtésére. A játéknak és a benne rejlő technikai megoldásoknak köszönhetően a projekt remélhetőleg nemcsak a programozás, hanem a kreatív tervezés területén is értékes tapasztalatokat fog nyújtani, amelyek tovább bővítik a játékipari ismereteimet.

Belső konzulens adatai	Külső konzulens adatai
Név: Hajdú Csaba	*Név: -*
Tanszék: Informatika Tanszék	*Munkahely: -*
Beosztás: egyetemi tanársegéd	*Beosztás: -*

Győr, 2025. 01. 03.

belső konzulens

külső konzulens

[A témát kiíró tanszék megnevezése, tanszékvezetőjének neve és aláírása]

Konzultációs lap Szakdolgozathoz

Hallgató neve		Balogh Attila
Hallgató Neptun kódja		HRFUT5
Tagozat (nappali/levelező)		nappali
Szak		Mérnök-informatikus
Specializáció		
Belső konzulens	Neve:	Hajdu Csaba
	Beosztása:	egyetemi tanársegéd
Külső konzulens (ha van)	Neve:	
	Beosztása:	
	Munkahelye:	
	Munkahely címe:	
	Telefonszáma:	

Dátum	Tevékenység	Konzulens aláírása
2025.02.19	Teendők átbeszélése, szakdolgozat struktúra tárgyalás	
2025.02.26	Első fejlesztések, tutorialok átnézése, Unity szintér	
2025.03.19	Activity diagram elkészítése, class diagram bontása	
2025.05.14	Első bemutató, architektúra átnézése, összegzése	
2025.09.29	Első pálya elkészítése, tesztelés	
2025.10.20	Mesterséges intelligencia fejlesztése	
2025.11.17	Dokumentáció átbeszélése, boss logika általánosítása	
2025.12.01	Végső dokumentáció, formázás javítása	

A Szakdolgozatot ellenőriztem, beadható/nem adható be.

Dátum:_____

Külső konzulens aláírása:_____

A Szakdolgozatot ellenőriztem, beadható/nem adható be; bírálatra bocsátható/nem bocsátható.

Dátum: 2025.12.03

Belső konzulens aláírása:_____

Nyilatkozat

Alulírott, Balogh Attila (HRFUT5), Mérnök Informatikus BSc szakos hallgató kijelentem, hogy a Legends of the Hero: A Platformer Adventure című szakdolgozat feladat kidolgozása a saját munkám, abban csak a megjelölt forrásokat, és a megjelölt mértékben használtam fel, az idézés szabályainak megfelelően, a hivatkozások pontos megjelölésével.

Eredményeim saját munkán, számításokon, kutatáson, valós méréseken alapulnak, és a legjobb tudásom szerint hitelesek.

Győr, 2025. 12. 04.

Balogh Attila

hallgató

Kivonat

[Legends of the Hero: A Platformer Adventure]

A szakdolgozatom témájának egy 2 dimenziós platformer típusú játék elkészítését választottam Unity játékmotor használatával. Az ilyen fajta játékok, a videójáték ipar egyik legrégebbi és legszélesebb körben elterjedt műfaját alkotják. A platformereknek megvannak a saját hagyományos, klasszikus mechanikái. Ezekhez hűen, ugyanakkor újfajta ötleteket is beleépítve szerettem volna elkészíteni a saját játékom.

A hagyományos platformer játékok lineáris és előre meghatározott viselkedésű ellenfeleket tartalmaznak, amelyek csökkentik a játék újra játszására irányuló vágyakat és a játékosokhoz való adaptációt. Felmerül tehát az igény egy olyan rendszer kialakítására, amelyben AI segítségével dinamikusan reagáló stratégiákat alakíthatunk ki, miközben megőrizzük a hagyományos platformer jellemzőket is.

A projekt tehát egy Unity alapú 2D platformer játékot valósít meg, amelyben a főellenfél viselkedését fuzzy logika szerű döntési mechanizmusok irányítják. Ez a megközelítés lehetővé teszi az ellenfél számára, hogy több, változó tényező alapján hozzon adaptív döntéseket, és ezáltal változatosabb és kiszámíthatatlanabb játékélményt biztosítson.

A játékmenet során a játékosnak különféle szobákon kell keresztül mennie, amelyek különböző akadályokat és ellenségeket tartalmaznak. Próbáltam úgy felépíteni a pályákat, hogy egymás után egyre nehezebb kihívások elé állítsák a játékost. Az utolsó szoba tartalmazza a játék legnagyobb próbatételét, a főellenség elleni harcot, amely adaptívan alkalmazkodik a játékos viselkedéséhez. A folyamatos előrehaladás során a játékos elsajátíthatja a sikeres végrehajtáshoz szükséges mechanikákat.

Szándékosan egy rage game típusú platformer játékot hoztam létre, de igyekeztem odafigyelni rá, hogy a frusztráció és a játék élvezete közötti egyensúly minél optimálisabb legyen. A játék nem csak szórakozást biztosít a játékosoknak, hanem számos képességük fejlődését is, például a türelem, a koncentráció vagy a reflexek fejlesztését. A tervezés és a fejlesztés során olyan tapasztalatokat szereztem, amelyeket úgy gondolom használni tudok a jövőben.

Abstract

[Legends of the Hero: A Platformer Adventure]

The topic of my thesis is the development of a 2D platformer game using the Unity game engine. This type of game represents one of the oldest and most widespread genres in the video game industry. Platformers are characterized by their own traditional, classic mechanics. My aim was to create a game that remains faithful to these conventions while also integrating new and innovative ideas.

Traditional platformer games often feature linear and predetermined enemy behavior, which reduces replayability and limits the game's ability to adapt to the player. This creates a demand for a system in which AI can be used to develop dynamically responsive strategies while still preserving the typical characteristics of the platformer genre.

The project implements a Unity based 2D platformer game, in which the behavior of the final boss is controlled by decision making mechanisms similar to fuzzy logic. This approach allows the enemy to make adaptive decisions based on multiple variable factors, providing a more varied and less predictable gameplay experience.

During the gameplay, the player must progress through several rooms, each containing different obstacles and enemies. The level design was structured to present increasingly difficult challenges in a sequential manner. The final room contains the ultimate challenge, a boss fight that adaptively responds to the player's behavior. Through continuous progression, the player is able to learn and master the mechanics required for successful completion.

The game was intentionally designed as a rage game style platformer, with particular attention paid to maintaining an optimal balance between frustration and enjoyment. In addition to providing entertainment, the game can also support the development of various player skills, such as patience, concentration, and reflexes. The design and development process also provided valuable experience that can be utilized in future projects.

Tartalomjegyzék

1. Bevezetés 1
2. Szakirodalmi kutatás: a platformer játékok és a játékmechanikák áttekintése 3
2.1. A játékmechanika és dizájn alapjai 3
2.2. A platformer játékok fejlődése és története 5
2.3. Pszichológiai hatások és játékélmény 7
2.4. Jövőbeli kilátások és fejlődés 8
2.5. Fuzzy logika és alkalmazása a játékfejlesztésben 9
3. Tervezés 11
3.1. Játék története 11
3.2. Követelmények 12
3.3. Használati esetek 12
3.4. Osztályok 14
3.5. Aktivitásdiagram, viselkedés leírása 19
3.5.1. Aktivitás diagram leírása: 19
3.6. Boss AI logika tervezés, különböző megoldások megvizsgálása 21
3.6.1. Első verzió: Fuzzy pontozás és adaptív memória 21
3.6.2. Második verzió: Prediktív, heurisztikus döntéshozás 23
3.6.3. Harmadik verzió: Kibővített fuzzy rendszer és adaptív komponensek 24
3.6.4. Összegzés 25
4. Implementáció, megvalósítás 26
4.1. Alapvető játékmechanika 26
4.1.1. Player logika 26
4.1.2. Ellenségek logikája 29
4.1.3. Csapda logikák 31
4.1.4. UI elemek logikája 33
4.1.5. Felvehető objektumok logikája 36
4.1.6. Kamera, szobák logikája 37
4.1.7. Logikák összefoglalása 39
4.2. Boss felépítése, működése 39
4.3. A végleges Boss AI logika működése 44
4.3.1. FuzzyLogic: az alap motor 44
4.3.2. BossAttackManager: a döntéshozó komponens 45
4.3.3. BossLearningMemory: adaptív tanulás és statisztika 46

4.3.4. BossTeleport: pozícióváltoztatás fuzzy alapon 46
4.3.5. BossRoomController: adaptív harci környezet 47
4.3.6. Az összetett logika működése 47
4.4. Tesztek 48
4.4.1. Play Mode tesztek 48
4.4.2. Edit Mode tesztek 49
4.5. Fejlesztéshez használt eszközök 50
5. Eredmények 51
5.1. Saját értékelés 51
5.2. Játékosok visszajelzései 52
5.3. Eltérés a tervtől 53
6. Konklúzió, jövőbelli tervek 55
6.1. Összegzés, értékelés, tapasztalatok 55
6.2. Jövőbeli tervek 55
Irodalomjegyzék 57

Ábrajegyzék

1. ábra: A játékmechanika komponenseinek fogalmi térképe 3
2. ábra: Super Mario Bros 5
3. ábra: A platformer játékok pszichológiai körfolyamata 8
4. ábra: Használati esetek 14
5. ábra: Tervezett osztálydiagram 18
6. ábra: A játék általános aktivitásdiagramja 20
7. ábra: Első AI működés logika 22
8. ábra: Második AI működés logika 23
9. ábra: Harmadik AI működés logika 25
10. ábra: Falra mászás 26
11. ábra: Player támadása 27
12. ábra: A checkpoint felvétel előtt 28
13. ábra: A checkpoint felvétel után 28
14. ábra: RangedEnemy 29
15. ábra: MeleeEnemy 29
16. ábra: Enemy BoxCast 30
17. ábra: Spikes 32
18. ábra: SpikeHead 32
19. ábra: Fire 32
20. ábra: FireTrap triggerelt állapotban 32
21. ábra: Saw 32
22. ábra: ArrowTrap 32
23. ábra: Boss HealthBar 33
24. ábra: DeathCounter UI 33
25. ábra: GameScore UI 33
26. ábra: GameOver Screen 34
27. ábra: PauseScreen 35
28. ábra: Main Menu Screen 35
29. ábra: HealtCollectable 36
30. ábra: Invulnerability 36
31. ábra: ExtraSpeed 37
32. ábra: PlusXP 37

33. ábra: Kamera pozíció 38
34. ábra: Boss megjelenése 40
35. ábra: Boss Osztálydiagram 42
36. ábra: A Boss működésének folyamatábrája 43
37. ábra: Fuzzy Tagsági függvények 44
38. ábra: PlayerMovement tesztek ..... Mellékletek
39. ábra: PlayerHealth Play Mode tesztek ..... Mellékletek
40. ábra: PlayerHealth Edit Mode tesztek ..... Mellékletek
41. ábra: DeathScreen ..... Mellékletek

1. Bevezetés

Az informatika egyik legsokoldalúbb területe a videójáték fejlesztés. Ennél a műfajnál ugyanolyan hatalmas jelentőséggel bír a programozás, a vizuális tervezés és a kreatív probléma megoldás is. Úgy gondolom egy megfelelően lefejlesztett játékkal nem csak egy technikai terméket kapunk, hanem egy olyan élményt is, amely különböző érzelmeket vált ki a játékosokból és képes a figyelem fenntartására egyaránt. A játékfejlesztésben a nyers kód, a grafika és a hanghatások nem különálló elemek, hanem ezek együttesen hoznak létre egy működő és élvezhető világot. Minden apróbbnak tűnő fejlesztési döntés hatással van a játékos élményére, legyen szó a karakter sebességéről, a kamerakezelésről vagy a pontozási rendszerről. Ennek a komplexitásnak és a sokoldalúságnak köszönhető, hogy ilyen izgalmas ez a terület.

Rengeteg különféle videójáték műfaj létezik és mindegyiknek sajátos logikája és tervezési elvei vannak. Míg egy stratégiai játékban például a döntéshozatal és az optimalizálás a legfontosabb, addig egy akciójátékban a játék ritmusa a fő szempont. Egy platformer játék esetében viszont a kulcs például a mozgás pontosságában vagy akár a kihívások fokozatos nehezedésében keresendő. Első ránézésre egy platformer játék nagyon egyszerűnek tűnhet, hiszen egy ilyen játékban csak ugrálni kell, haladni előre és kikerüli az akadályokat. Azonban ez egy nagyon érdekes játéktípus, hiszen a részletek precíz kidolgozásán múlik, hogy az adott játék élvezetes lesz vagy pedig frusztráló. Sokféle dologtól függhet, hogy egy játékos miként ítél meg egy játékot. A legfontosabb ilyen tényezők talán a pályák hangulata, a karakterek mozgásainak természetessége, valamint a játék hangalapú és vizuális visszajelzései.

A szakdolgozatom során Unity játékmotor felhasználásával egy ilyen 2D platformer játékot hoztam létre. Ezen játékmotor kiválasztása tudatos döntés volt, mert úgy gondolom, hogy ez egy stabil, jól dokumentált és aktív közösséggel rendelkező rendszer, amelyben minden megtalálható vagy beépíthető, ami egy platformer játék fejlesztése során szükséges lehet. Ilyen fontos Unity-s komponensek, amiket felhasználtam például a *Rigidbody2D*, *Collider2D*, vagy akár az *Animator*. A fejlesztés során C# programnyelvet használtam a scriptek megírásához és igyekeztem, hogy minden mozgás, interakció kézzel hangolt, folytonos tesztelési tapasztalataimnak megfelelően kialakított legyen.

A játékom fő koncepciója az egyre nehezedő kihívásokra és a játékos ezen kihívások során elért fejlődésére épül. Az egymás után következő szobák különféle akadályokat és ellenségeket rejtenek, amelyek legyőzéséhez a játékosnak újabb és újabb mechanikákat kell elsajátítani vagy

fel kell használnia a már addig megszerzett tapasztalatait. A játék csúcspontja és egyben finálója egy boss fight, amelynek fejlesztése során mesterséges intelligencia szerű logikát hoztam létre. A boss felismeri a játékos viselkedését és ehhez igazodva választja ki a támadásait és a taktikáját. Ezzel azt szerettem volna elérni, hogy a harc ne ugyanúgy egy beadott minta alapján ismétlődjön, hanem minden alkalommal új stratégiát igényeljen.

A játék fejlesztése során számos kihívásba ütköztem. Az egyik legidőigényesebb dolog például az ugrás mechanikájának a finomhangolása volt, amelyet hosszas tesztelési folyamat után, de véleményem szerint sikeresen be tudtam hangolni úgy, hogy az a megfelelő, természetes érzést keltse. A pályatervezés során is rengeteg tesztelésen estem át. Különböző blokkelrendezéseket és ellenfél pozíciókat próbáltam ki mire megtaláltam a megfelelőnek bizonyuló egyensúlyt a nehézség és a sikerélmények elérésének biztosítása között. Ezek a döntések aprónak tűnhetnek, de a tapasztalataim alapján egy platformerben minden pixelnyi mozgás számít.

A mesterséges intelligencia beépítése a játékba új kihívások elé állított a fejlesztés, sőt már a tervezés során is. Ahogy azt már említettem is a MI-t a boss elleni harcnál használtam, mégpedig nem kifejezetten a mozgás irányítására, hanem inkább döntési logikák beépítésére. Ezen logikák a játékos viselkedése alapján módosítják a boss stratégiáját és taktikáját. Ennek a célja nem a nehézségi szint túlságosan elrugaszkodott megnövelése volt, hanem a játék újrajátszhatóságának, dinamikusságának és adaptivitásának a fokozása. Egy adaptív logika bevezetése egy új színt visz a játékba, hiszen így a harc mindig más kihívást jelent és ez jelentősen növeli a játék élettartamát és megítélését.

A platformer játékok pszichológiai hatása is különleges. A hibázás és az újra próbálkozás ciklusa fejlődést biztosít a játékosnak, ami, ha minden megfelelően van finom hangolva, erős motivációt tud nyújtani a játék során. Minden kisebb győzelem, akár egy akadály leküzdése vagy egy nehezebb ugrás sikeres végrehajtása fokozatosan alakítja ki a játékos magabiztosságát. Az úgynevezett flow állapot támogatja ebben a játékost, ami azt jelenti, hogy teljesen belemerül az adott tevékenységbe és olyan feladatokat hajt végre, amelyek elég nehezek ahhoz, hogy izgalmasok legyenek, de frusztrálóak viszont ne.

A dolgozatom célja tehát, hogy bemutassa az ötlet megszületésétől, a tervezésen át, a végleges megvalósításig egy 2 dimenziós platformer játék fejlesztési folyamatát. Ismertetem a főbb technikai megoldásokat, a Unity eszköztárának számomra hasznos részeinek a használatát, valamint a játékmechanikai és tervezési döntéseket, amelyek a végeredményhez vezettek. Az elkészült játék példát mutat arra, hogyan lehet ötvözni a klasszikus játékmenetet a modern fejlesztési technikákkal, és miként válhat az MI a játékmenet aktív, reagáló szereplőjévé.

2. Szakirodalmi kutatás: a platformer játékok és a játékmechanikák áttekintése

2.1. A játékmechanika és dizájn alapjai

A videójátékokban a játékmechanika az egyik legmeghatározóbb, hiszen ezen múlik, hogy a játékos milyen módon és mennyire kapcsolódik a virtuális világhoz, valamint, hogy az interakciók milyen szabályszerűségek mentén zajlanak. Ennek megfelelően tehát, ez egy olyan fogalom, ami magába foglal a mozgási és cselekvési lehetőségeken keresztül, a játékszabályokon át, az ezekből fakadó visszacsatolásokig mindent **[3]**. Egy platformer játéknál ez leginkább a gravitációs megoldásokhoz és a különböző szintek szerkezetének kialakításán keresztül az adott akadályok leküzdéséhez és ehhez kapcsolódóan a karakterek mozgásához köthető. Az alapvető logikák, mint az ugrás és a futás, olyan technikai megoldások, amelyek a játék alapritmusát adják és leginkább befolyásolják a játék természetességének, élvezhetőségének a mértékét. A mechanika tehát nem csak a játék szabályait határozza meg, hanem a játékos tapasztalatait is (**1. ábra**).

1. ábra: A játékmechanika komponenseinek fogalmi térképe

Katie Salen és Eric Zimmerman *Rules of Play: Game Design Fundamentals* című alkotása a játékmechanika fogalmát a játékrendszerek működésének alapjaként definiálja. Az alkotók kihangsúlyozzák, hogy szerintük a játéktervezés legfontosabb szempontja a játékosok élményeinek az előtérbe helyezése és ennek megfelelően a mechanika tudatos kialakítása. A visszajelzések, szabályszerűségek, mechanikák nyilvánvalóan nem önmagukért léteznek, hanem mindezek közösen járulnak hozzá az interaktív élmények szolgáltatásához **[3]**. A platformer műfaj kifejezetten jó példaként szolgál erre, mivel ezen játékok esetében a

legapróbbnak tűnő mozgási részletek, például az ugrások sebessége vagy az időzítések pontos kialakítása közvetlenül befolyásolják az élmény intenzitását. Egy tökéletesen beállított animáció vagy mozdulat teljes mértékben megváltoztathatja a ritmust és a játék befogadhatóságát.

A platformer játékokban az egymásra épülő szintek megtervezése, leginkább egyensúlykeresés a vizuális esztétika, a mechanikai kihívások és az ezekből következő tanulási ív között. Nem csak látványban különböznek egymástól a pályák, hanem a sikeres teljesítés érdekében, fokozatosan teszik próbára a játékos képességeit. Az újabb akadályok megjelenése, vagy a már sikeresen teljesített akadályok összetettebb módon megjelenítése és az apró nehézségi emelkedések egyre összetettebb mozgásmintákat igényelnek a játékosoktól, amelyek létrehoznak egyfajta tanulási folyamatot, amely a játékos fejlődését biztosítja **[2]**. A játékos így megtanulja helyesen használni a mozgásfizikákat, felismeri a pályastruktúrák mintázatait és kialakíthatja saját sikerre vezető taktikáit, stratégiáit. Ez a tanulás tapasztalatok útján történhet, ami elmélyítheti az élményt és remélhetőleg fenntartja az érdeklődést is hosszabb távon. Egy jól összekomponált játékmenet tehát nem igazán a nehézség folyamatos növeléséről szól, hanem annak dinamikusságáról és megfelelő ütemezéséről **[5]**.

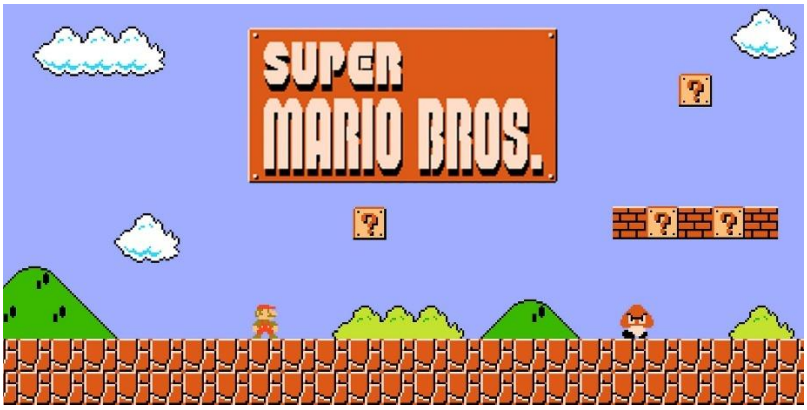
A játékmechanika élményszerű oldalát, megközelítésit Steve Swink „game feel" koncepciója taglalja **[4]**. Ez a fogalom a fizika, az animációk és a visszajelzések együttes hatásaként írható le. Fogalmazhatunk úgy is, hogy egy játékot akkor „jó érzés" játszani, amikor a mozdulatok gördülékenysége, az irányítási mechanikák pontossága és a játékvilág minél élőbb hatásának beállítása egyaránt fellelhető a játékban. Ha például a főhős karakterének mozgása, ugrásának íve vagy akár a talajra érkezés megjelenítése megfelelően finomhangolt, akkor a játékos szinte már fizikailag is érzékelheti ezeket. A „game feel" valójában tehát az érzelmi elmélyülés egyik legfontosabb eszköze, nem csupán egy technikai minőség.

Ezek az érzéki visszacsatolások szorosan összeköthetőek Csíkszentmihályi Mihály flow elméletével. Eszerint egy jól megtervezett és kivitelezett mechanikákkal ellátott játék képes fenntartani azt az állapotot, amelyben a játékos figyelme teljesen az adott cselekvésre tud összpontosulni, mialatt az időérzéke részben megszűnik. A mozgási ritmusok, a kihívások fokozatos nehezedésének dinamikája és a játék visszajelzései együttesen teremthetik meg ezt a fajta flow élményt. Ha a rendszer túl egyszerű, akkor esélyes, hogy unalmat okoz, viszont, ha túl nehéznek bizonyul, akkor túlzott frusztrációt szülhet. Ennek az optimális egyensúlynak a megtalálása a játékmechanikai tervezés egyik legnagyobb kihívása **[1][[4]**.

2.2. A platformer játékok fejlődése és története

A platformer játékok története szorosan összefonódik a videójáték ipar fejlődésével, hiszen ez a műfaj akkor kezdett kibontakozni, amikor az 1980-as évek elején a digitális szórakoztatás gyors ütemben terjedni kezdett. Ekkor a fejlesztők új módokat kerestek minél interaktívabb élmények nyújtására. A korszakban jelen lévő technikai korlátok ellenére a kreatív ötletek mentén történt játékmechanikai újítások látványos fejlődést indítottak el. A *Donkey Kong* (Nintendo, 1981) például már tartalmazta a platformer műfaj alapvető elemeit, mint a több szintből álló játékszerkezetet, az akadályok elkerülésének mechanikáját és az ugrálást, mint fő platformer mozgásfajtát. Ez a játék tekinthető az első jelentős platformernek, amely megalapozta a későbbi fejlesztések irányát és logikáját **[11]**.

A következő nagy szintlépést a *Super Mario Bros* (**2. ábra**)[1]. által 1985-ben kiadott Nintendo Entertainment System (NES) konzol megjelenése jelentette. Ennek az eszköznek a létrehozása egy új szintre emelte a játéktervezést. A precízebb irányítás, a karakteres vizuális világ és a fokozatosan nehezedő pályák együttesen egy addig még nem tapasztalt játékélményt hoztak létre. A *Super Mario Bros.* nem csak technológiai, hanem kulturális mérföldkővé is vált. Jeff Ryan *Super Mario: How Nintendo Conquered America* című könyve részletesen tárgyalja, miként formálódott át az egész videójáték kultúra ezen játék sorozat miatt és hogyan járult ez hozzá a Nintendo globális sikeréhez **[10]**. A *Super Mario* sorozat egyértelműen meghatározta azt az irányt, ami megmutatja, hogy mi kell egy „jó platformer" játékhoz. Leginkább egy jól megérthető szabályrendszer, minél finomabbra hangolt mozgásmechanikák és egy fokozatos tanulási ív **[3]**.



2. ábra: Super Mario Bros

A Nintendo mellett egy másik kulcsszereplő a platformerek fejlődésében a Sega volt. Az általuk 1991-ben kiadott *Sonic the Hedgehog* a *Mario* sorozat játékainak megfontoltabb, precízebb

[1] https://www.nintendo.com/en-gb/Games/NES/Super-Mario-Bros-803853.html?srsltid=AfmBOoqI2H1D5kmhytMpZFxW9A7zazpCssOtNrh29QWle6eIyNzZubMD

ritmusával szemben a gyorsaságra és a kinetikus energiára helyezte leginkább a hangsúlyt **[9]**. A sebességre épülő *Sonic* sorozat játékaiban tehát a lendület és a fizikai dinamika váltak a fő szervezőelemekké. Ez a fajta megközelítés új dimenziót adott a műfajnak, hiszen az ilyen platformerek nem csupán az ugrások és akadályok sorozati lettek, hanem egy folyamatos mozgásélményt alakítottak ki, ahol a játékos tempóérzékére és reflexeire került inkább a hangsúly. A Sega ezzel egy saját stílusú, versenyképes alternatívát hozott létre a Nintendo domináns irányával szemben, ami egy új lendületet adott az iparnak.

A technikai ugrás az 1990-es évek végén és a 2000-es évek elején egy új korszakot nyitott meg a műfaj számára. Ekkor alakult ki a háromdimenziós grafika, amelynek térhódítása alapvetően változtatta meg az irányítási rendszereket és a játékmenet logikáját. A 3D platformerek úttörői, mint például a *Super Mario 64* (1996), a *Crash Bandicoot* (1996)vagy akár a *Spyro the Dragon* (1998) megmutatták, hogy miként lehet beintegrálni a játékmechanikába a térbeli mozgást és ezt a fajta kamerakezelést **[8]**. A *Super Mario 64* még ezek közül is kiemelkedő volt, hiszen meghatározó szerepet játszott az analóg karos játékok irányításának bevezetésében és a szabadon mozgatható kamera koncepciójának a kialakításában, ami azóta is alapvető elem a 3D-s játékok tervezésénél **[6]**. A fejlesztők számára ezek az új eszközök egyszerre nyújtottak nagyobb szabadságot és kihívást is.

Az úgynevezett indie játékfejlesztés térnyerése a 2000-es évek végén új energiát vitt a platformerek világába. Ebben a fejlesztési irányzatban a fejlesztők kis költségvetéssel, de erős kreatív meglátásokkal tértek vissza a kétdimenziós forma gyökereihez. Ekkor jelentek meg olyan játékok, mint a *Braid* (2008), a *Super Meat Boy* (2010) vagy a *Celeste* (2018). Ezekkel a játékokkal a fejlesztők a klasszikus platformer mechanikákat modernebb technikai környezetbe ültették át, miközben igyekeztek új narratív és érzelmi mélységeket is hozzáadni a játékmenethez **[7]**. Ezek a játékok egyszerre voltak nosztalgikusak és egyszerre mozogtak az innováció határán is, így megmutatva, hogy a platformer műfaj nem csupán a múltidézésre alkalmas, hanem folyamatosan megújuló, élő kísérleti terep is lehet. Ez az indie hullám ráadásul egy újfajta érzékenységet is behozott a tervezésbe, hiszen igyekeztek személyesebb, érzelmileg gazdagabb és esztétikailag is tudatosabb játékokat kialakítani.

A platformer tehát a videojáték történet egyik legdinamikusabban fejlődő műfaja, amely képes volt minden technológiai korszakban újra értelmezni önmagát. A korai 2D-s játékok precizitásától a háromdimenziós világok szabadságáig, majd az indie reneszánsz introspektív, minimalista megoldásokig ível a története. Ez a folytonos megújulás a műfaj egyik legfőbb ereje. A platformer egyszerre őrzi a klasszikus játékmenet örökségét és szolgál a kísérletezés

terepéül a játékfejlesztők számára.

2.3. Pszichológiai hatások és játékélmény

A platformer játékok esetében a pszichológiai hatások rendkívül jelentősek és érdemes rájuk komoly figyelmet fordítani. Ez egy kulcsfontosságú része a játékmenet és a dizájn megértésének, hiszen a játékélmény szorosan kapcsolódik a játékos mentális és érzelmi folyamataihoz. A játék során bekövetkező elmélyülés mértéke és az, hogy a játékos hogyan értékeli a játékot, jelentős mértékben függ attól, hogy a játékosnak milyen a motivációja, a figyelme és hogy milyen érzelmi reakciókat él át a játék során. Jamie Madigan *The Psychology of Video Games* című munkájában részletesen elmagyarázza a videójátékok pszichológiai aspektusait, ide sorolva a szórakoztatást, a jutalmazást és ezek által a figyelem tartós fenntartását és a tanulási metódusokat **[13]**. A platformer játék típus különösen alkalmas a pszichológiai hatások vizsgálatára, mivel az itt használt egyszerűnek tűnő, azonban kifinomult mechanikák, a folyamatos kihívások elé állítás és a szinte azonnali visszacsatolások intenzív mentális aktivitást váltanak ki a játékosból és így tartják fent a figyelmet és érik el az elmélyülés érzését.

A platformerek mechanikai elemei a játékos kognitív képességeit is erősen igénybe veszik és próbatételek elé állítják. Egy ilyen játék során mind a problémamegoldás, a gyors döntéshozatal, a reakcióidő és a szem-kéz koordináció fontos részei a sikerességnek, főleg a bonyolult akadályoknál vagy egy időzített mozdulat végrehajtása esetén **[12]**. A játék tanulási folyamata ciklikus próbálkozásokból és hibázásokból áll, így elérve a természetes tanulási formát. A játékos előre haladása során folyamatosan fejleszti képességeit, miközben új stratégiákkal kísérletezhet és sikerélményeket szerezhet, jó esetben anélkül, hogy túlzott mentális túlterhelésen menne keresztül. Ez a mechanizmus erősíti a játékélményt, de ezen felül hozzájárul hosszú távon a kognitív képességek és a figyelem fókuszálásának fejlődéséhez.

Madigan írásaiban kihangsúlyozta, hogy a videójátékok egyik legjelentősebb pszichológiai vonzereje a jutalmazási rendszer, amelyet pontszámok gyűjtésével, új szintek elérésével vagy akár vizuális visszajelzésekkel biztosíthat egy játék **[13]**. Ezek a megerősítési mechanizmusok dopamintermelést idéznek elő az agyban, ami a motiváció növelését és az érdeklődés fenntartását váltja ki a játékosokból. A platformerek esetében ezek a jutalmazások szorosan kapcsolódnak a tanulási fázishoz is, hiszen így a játékos nem csupán új lehetőségeket kap, hanem egyre ügyesebbé válhat a játékon belüli szabályrendszer kezelésében és képes lesz összetettebb mozdulatok végrehajtására is. A jutalom szerzés és a fejlődés így párhuzamosan zajlik, erősítve ezzel a játék élvezeti értékét és a kompetenciaérzetet (**3. ábra**).

3. ábra: A platformer játékok pszichológiai körfolyamata

Egy ilyen fajtájú játékélmény tehát egyszerre igényel kognitív, érzelmi és viselkedési követelményeket is. Az elért sikerek, például egy nehezebb akadály leküzdése, növelhetik a játékos hatékonyság érzetét, míg a saját teljesítményével való elégedettség érzet növeli a játékos önbizalmát **[14]**.

2.4. Jövőbeli kilátások és fejlődés

A platformer játékok műfaja hosszú és változatos utat tett meg a kezdeti, kétdimenziós, pixeles világtól az egészen komplex, háromdimenziós megoldásokig. A különféle technológiák fejlődése és kialakulása nem csupán a vizuális megjelenésben hozott újításokat, hanem a játékmenet teljes struktúrájában reformokat idézett elő. Lehetővé vált finomabb mozgásmechanikák, dinamikusabb pályaszerkezetek és részletesebb, kifinomultabb interakciók beépítése a platformerekbe. Várhatóan a jövőbeli fejlesztések során a platformerek továbbra is az éppen aktuális, modern technológiai trendekhez fognak igazodni, miközben megőrzik az egyszerű mechanikai megoldásokban rejlő játékélményt. Az innováció egyik kulcsa az, hogy klasszikus elemeket mindig kreatívan újra értelmezve, de megtartsák a játékokban, így a műfaj mindig képes lesz megújulni **[15]**.

A videójátékipar egyértelműen az újabb élményalapú interakciókat nyújtó irányvonalak felé látszik elmozdulni, vagyis a virtuális valóság (VR) és a kiterjesztett valóság (AR) kaphat leginkább szerepet a jövőben. Ezek az újfajta technológiák lehetőséget kínálnak arra, hogy a térbeli érzékelést és a valós emberi testmozgást bele lehessen integrálni egy platformer környezetbe, ezzel növelve az immerszív élmény intenzitását is. A VR és AR platformer megoldások tervezése új típusú megközelítést és gondolkodást igényel, hiszen ezekben az esetekben a játékos nem csak interaktív elemeket kezel, hanem fizikailag is jelen van a digitális

játéktérben, ami teljesen új lehetőségeket nyit meg a pályák, akadályok és a mechanikák kialakításában is **[16]**.

Egy figyelemre méltó példa a VR alapú platformerekre az *Astro Bot Rescue Mission* (Sony, 2018), amely sikeresen demonstrálta, hogy a térbeli navigáció és a játékos nézőpontjának ilyen fajta manipulálása valóban teljesen új játékmeneti mechanizmusokat és élményeket hozhat létre. A VR eszköz lehetővé teszi, hogy a játékos valóban benne legyen a digitális környezetben, ne csak irányítsa azt, ezáltal egy sokkal intenzívebb és személyesebb játékélmény érhető el. Jerald szerint azonban, amekkora előny, akkora hátrány is, hogy a felhasználó aktív résztvevőként érzékeli a digitális világot, hiszen ez megint egy teljesen más, komplexebb szemléletet követel meg a fejlesztőktől **[16]**.

A jövő platformer játékaiban továbbá valószínűsíthetőleg egyre gyakrabban fellelhetőek lesznek a mesterséges intelligencia (AI) alapú és a gépi tanulás alapú megoldások is **[17]**. Ezek szintén teljesen újfajta lehetőségek kihasználását teszik lehetővé. A procedurális pályatervezés és a dinamikusan vagy akár adaptívan változó nehézségi szintek területén a legígéretesebbek ezek a technológiák. AI segítségével képesek lehetünk ugyanis olyan játékmeneteket létrehozni, amelyek adaptívan, a játékos szokásaihoz, viselkedéséhez és képességeihez igazítják a kihívásokat. Ez a személyre szabott megközelítés, amellett, hogy intenzívebbé teszi a játékélményt, segít a flow kialakításában és a játékos elmélyültségének a fenntartásában is.

Egy másik jövőbeli megközelítés lehet például, a felhasználói élményekhez kapcsolódóan, a hozzáférhetőség (accessibility) szempontja, különösen a fizikai vagy mentális nehézségekkel élő játékosok számára. Várhatóan a jövő platformerei nagyobb figyelmet fordítanak az efféle inkluzív dizájnra és kialakításra, hogy a műfaj szélesebb közönség számára is élvezhető lehessen. Ez a fajta szemlélet nem csak a játékos bázist bővítené ki, hanem új tervezési kreativitást igényelhetne, hiszen a különböző képességekkel rendelkező játékosok eltérő interakciókat igényelnek **[18]**.

2.5. Fuzzy logika és alkalmazása a játékfejlesztésben

A fuzzy logikával a klasszikus, bináris logikától eltérően, ahol csupán csak „igaz” vagy „hamis” értékekkel lehet dolgozni, lehetőséget kapunk a részleges igazságok kifejezésére is **[19]**. Ez kifejezetten hasznos lehet olyan rendszereknél, ahol a változók értékei ténylegesen elég sűrűn változnak és nem is éles határokkal rendelkeznek. A mesterséges intelligenciát integráló játékokban gyakran alkalmazzák a fuzzy logikát. Leginkább ellenségek viselkedésének modellezésére, döntéshozatalra vagy adaptív nehézség kialakítására alkalmas ez a logika. A hagyományos állapotgépekkel ezt csak túl komplex és merev megoldásokkal lehetne hasonlóan

reprezentálni.

A fuzzy logika alapja egy szabály alapú rendszer, amelyben tagsági függvényeket felhasználva értékeljük a bementei változókat **[20]**. Ezek a tagsági függvények minden változóhoz hozzárendelik azt a mértéket, hogy mennyire tartozik egy adott kategóriába, például, hogy a „közel”, a „közepes” vagy a „távol” kategóriába tartozik a játékos és az ellenség közötti távolság. A bementi paraméterek kiértékelése tehát szabályok alapján történik, ahogy azt már említettem és ez általában így néz ki: „HA <feltétel> AKKOR <akció>”. Például, ha a boss karakterét nézzük egy szabály így nézhet ki: „HA a játékos távolsága kicsi ÉS a boss életereje magas, AKKOR támadj gyorsan”. Ezen szabályok teszik lehetővé, hogy mondjuk egy boss viselkedése rugalmasan alkalmazkodjon a játékos cselekedeteihez és ne csupán előre meghatározott mintákat kövessen.

A fuzzy szabályok kiértékelése után a rendszer defuzzifikációt alkalmazhat, hogy a fuzzy értékeket konkrét numerikus kimenetté alakítsa **[20]**. Ennek a lépésnek köszönhető, hogy a karakter a játékban valós, végrehajtható akciókat csináljon, például támadjon, visszahúzódjon vagy éppen mozogjon egy adott irányba. Többféle defuzzifikációs módszer létezik, mint például a centroid vagy a maximum membership és ezek biztosítják, hogy az AI döntések egyszerre legyenek logikusak, kiszámíthatóak és a játékos számára életszerűek.

A fuzzy rendszerek egyik előnye, hogy a karakter viselkedése mindig koherens marad a játék logikájával, oly módon, hogy az mégis kevésbé kiszámítható **[19].** Ez szintén hozzájárul a korábban már tárgyalt flow élményhez, hiszen a játékosnak folyamatosan alkalmazkodnia kell az ellenség aktuális taktikájához, cselekedeteihez és így a harc nem válik kiszámíthatóvá vagy monotonná. A fuzzy logikával emellett lehetősége van a fejlesztőknek a játékmechanikák finomhangolására, ugyanis a tagsági függvények és a szabályok módosításával a nehézség, az ellenség reakcióideje vagy a stratégiák komplexitása tetszőlegesen állítható.

3. Tervezés

3.1. Játék története

A játék története egy teljesen fiktív, kitalált világot és eseménysort ábrázol, amely kizárólag a játék élményének fokozását szolgálja. A történet célja, hogy színesebbé, izgalmasabbá tegye és elmélyítse a játékosok élményét, miközben nem tükrözi a valóságot vagy nem kapcsolódik semmilyen történelmi eseményhez:

Egy távoli királyság, amelyet hosszú évek óta béke és boldogság jellemzett, most sötét árnyékba borul. A birodalom trónján ülő bölcs és igazságos király és felesége, a királynő, egy napon eltűntek. A királyságot egy sötét varázsló, Zorlok, hódította meg, aki hatalmas mágiájával uralma alá kerítette az egész vidéket.

Zorlok varázslatai és sötét ereje hamarosan átformálták a királyságot. Az egykor virágzó földek most füstölögtek, az erdők kihaltak, és a városok romokban hevertek. Az emberek elkeseredtek, és az egyetlen reményük egy hősben rejlik, akit a legendák szerint a királyság megmentésére választottak ki. A hős, akinek a nevét már régóta mesélik a tűz körül, aki egyedül képes lehet szembeszállni Zorlok hatalmával, és visszanyerni a birodalom békéjét.

A játékban a hős útja nemcsak egy fizikai, hanem egy lelki kaland is, ahol nemcsak a sötét mágiát kell legyőznie, hanem a belső félelmeit és kétségeit is. Ahogy előrehalad a pályákon, különféle kihívásokkal találkozik, amelyek próbára teszik bátorságát, gyorsaságát és intelligenciáját. Az akadályok és ellenségek egyre veszélyesebbek, miközben a hős egyre jobban megtanulja, hogyan kell kihasználni környezetét, és hogyan tudja legyőzni azokat, akik megpróbálják megállítani őt.

A hős nemcsak a harcra koncentrál, hanem azon is dolgozik, hogy felfedezze a királyságot, és megértse Zorlok erejének forrását. Minden egyes lépésével egyre inkább nyilvánvalóvá válik, hogy a varázsló ereje nem csupán a mágiából származik, hanem a birodalom népének félelméből és szorongásából táplálkozik. A hősnek tehát nemcsak a fizikai erőre, hanem a szellemi és érzelmi erőre is szüksége van, hogy legyőzze Zorlokot.

A végső összecsapás előtt, amely egy elpusztított erdőben zajlik, a hős minden eddigi tudását és tapasztalatát fel kell használja, hogy legyőzze a varázslót és annak különböző sötét varázslatait. Zorlok nemcsak egy egyszerű ellenfél, hanem egy igazi stratégikus gondolkodó, aki a hős minden mozdulatát figyeli és próbálja manipulálni a környezetet, hogy a hőst csapdába ejtse.

A hős számára az igazi kihívás nemcsak a harc, hanem a pszichológiai küzdelem is. Mivel Zorlok képes a hőst is kétségbe vonni, hogy vajon elég erős-e, hogy megmentse a királyságot,

és hogy valójában nem ő az, aki a végső csapást mérheti a varázslóra. A történet végén a hősnek a saját belső félelmeit és gyengeségeit is le kell győznie, hogy véglegesen legyőzze Zorlokot, és elhozza a királyság számára a várva várt szabadságot.

3.2. Követelmények

Az alábbi táblázat a platformer típusú játékom fejlesztése során megfogalmazott legfontosabb funkcionális követelményeket tartalmazza (**1. táblázat**). A táblázat célja, hogy rendszerezetten bemutassa a játék egyes jellemzőit, valamint ezek fontossági sorrendjét a játékélmény és a megvalósítás szempontjából. A követelmények meghatározása segítette a fejlesztési folyamat strukturálását, és alapot nyújtott a további tervezési és implementációs lépésekhez.

ID	Eset	Leírás	Prioritás
1.	Szobák bejárása	A történet főszereplője szobáról szobára vándorol és harcol a fellépő akadályok ellen.	Magas
2.	Játékos mozgása	A játékos tudja mozgatni a főszereplőt a billentyűzet segítségével.	Magas
3.	Játékos ugrása	A játék egyik kulcsfontosságú irányítási mechanikája az ugrás, hiszen erre sűrűn lesz szükség a pályán való túléléshez.	Magas
4.	Környezeti interakció	A játékos cselekedetei kihatással vannak a környezetére és ugyanúgy fordítva is.	Közepes
5.	Történethez hű ábrázolás	A játékos a szobákon való folyamatos áthaladással egyre közelebb kerül a küldetés, vagyis a játék véghezviteléhez.	Alacsony
6.	Ellenségek megjelenése	A játékos a történet előrehaladásával párhuzamosan egyre nehezebben legyőzhető ellenfelekkel találkozik.	Közepes
7.	Boss elleni küzdelem	A játék utolsó nagy megmérettetése egy AI segítségével működő főellenség legyőzése.	Magas
8.	Akadályok megjelenése	Az egyes szobákban a legyőzendő ellenségek mellett különböző akadályok kikerülésével, kijátszásával kell megbirkóznia a játékosnak.	Magas

1. táblázat: Követelmények

3.3. Használati esetek

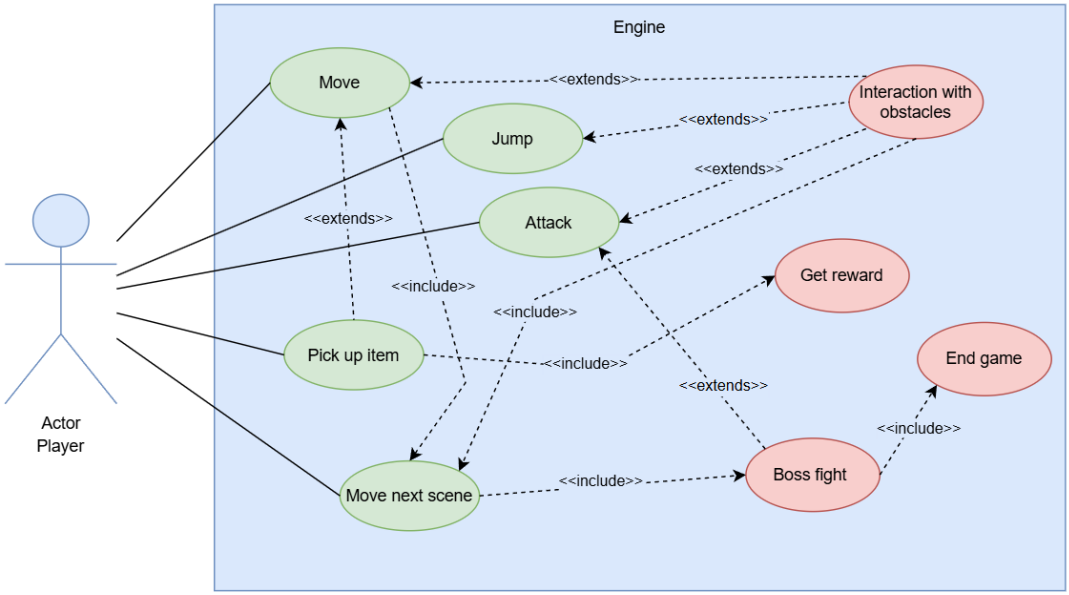
Az alábbi táblázat a platformer játékom használati eseteit mutatja be, amelyek a játékos és a játék közötti interakciók alapját képezik (**2. táblázat**). A táblázat célja, hogy részletesen ismertesse azokat a cselekvéseket és folyamatokat, amelyeket a játékos végrehajthat, illetve

amelyek a játék során automatikusan bekövetkeznek. Ezek az esetek segítenek meghatározni a játékmenet logikáját, és irányt adtak a fejlesztés során megvalósítandó funkciók kialakításához. Az egyes elemek prioritása a játékélmény szempontjából betöltött fontosságuk alapján került meghatározásra.

ID	Eset	Leírás	Prioritás
1.	Move (player)	A karakter mozgása az alapvető játékmeneti elem. A játékos lehetőséget kap arra, hogy balra vagy jobbra mozogjon, esetleg más típusú mozgásokat is végrehajtson, mint például futás, guggolás, vagy falon mászás.	Magas
2.	Jump (player)	Az ugrás a platformer játékok egyik legismertebb mechanikája. A játékosnak különböző típusú ugrásokat kell végrehajtania, hogy elérje a magasabb helyeket, megkerülje az akadályokat.	Magas
3.	Interaction with obstacles (game)	A platformer játékok szinte minden szintjén találkozhatunk akadályokkal, amelyek lehetnek statikusak vagy dinamikusak. A játékosnak ezeket ügyesen kell átlépnie vagy elkerülnie, hogy tovább haladjon	Magas
4.	Attack (player)	A platformer játékokban gyakran előfordulnak ellenségek, amelyeket a játékosnak el kell kerülnie vagy legyőznie.	Magas
5.	Pick up item (player)	A gyűjtögetés a platformer játékok egyik alapeleme, amely nemcsak a játékos motiválására szolgál, hanem további jutalmakat is adhat.	Közepes
6.	Get reward (game)	Ha a játékos felvesz egy különleges képességet adó elemet, akkor jutalomban részesül	Közepes
7.	Move next scene (player)	A játékosnak adott akadályok és ellenségek legyőzése után lehetősége van tovább haladni újabb akadályokhoz.	Magas
8.	Boss fight (game)	A játék utolsó nagy megmérettetése egy boss legyőzése.	Magas
9.	End game (game)	A boss legyőzése után a játékos eléri végső célját és véget ér a játék.	Közepes

2. táblázat: Használati esetek

A használati esetekhez készítettem egy diagrammot is, amely vizuálisan szemlélteti a játékos és a rendszer közötti legfontosabb interakciókat **(4. ábra)**. A diagram célja, hogy áttekinthető formában mutassa be, hogyan lép kapcsolatba a játékos a játékkal, milyen főbb tevékenységeket hajt végre, és ezek hogyan kapcsolódnak egymáshoz. Ez az ábra nemcsak a fejlesztési folyamatot segítette, hanem a rendszer működésének gyors megértését is támogatja.



4. ábra: Használati esetek

3.4. Osztályok

A tervezés következő lépése az osztályok megtervezése volt (**5. ábra**). Fontos megjegyezni, hogy ezek csak tervek voltak, a megvalósítás során ettől több helyen is eltértem ahogy az az implementációs fejezetben látható.

1. Entity (Absztrakt) osztály

Az Entity osztály egy absztrakt osztály, amely az összes entitás alapvető megkülönböztető tulajdonságait tartalmazza, ilyen lehet az ID és a name.

Attribútumok:

- **id**: Az objektum egyedi azonosítója. (int)
- **name**: Az objektum neve. (string)

Metódusok:

- **-**

2. LivingEntity (Absztrakt) osztály

Az LivingEntity osztály egy absztrakt osztály, amely összefoglalja azokat a közös tulajdonságokat és metódusokat, amelyek több osztályban is előfordulnak. A Hero, Enemy és Boss osztályok mind az Entity osztályt öröklik, és implementálják a szükséges viselkedéseket.

Attribútumok:

- **id**: Az objektum egyedi azonosítója. (int)
- **name**: Az objektum neve. (string)
- **health**: Az objektum életereje. (int)
- **damage**: Az objektum sebzése. (int)
- **speed**: Az objektum mozgási sebessége. (int)

Metódusok:

- **move()**: A mozgás vezérlése. A konkrét osztályoknak kell implementálniuk.
- **takeDamage(**amount**)**: A sebzés kezelése. A konkrét osztályoknak kell implementálniuk, és csökkenti az életerőt a kapott sebzéssel.
- **attack()**: Támadás végrehajtása egy másik Entity objektumra. Csökkenti a cél életerejét a támadó sebzésével.

3. Hero osztály

A hős osztálya tartalmazza a karakter különböző tulajdonságait, mint például az életerő, sebzés, mozgás és egyéb képességek.

Attribútumok:

- **id**: A főhős azonosítója. (int)
- **name**: A főhős neve. (string)
- **health**: A hős életereje. (int)
- **damage**: A hős által okozott sebzés. (int)
- **speed**: A hős mozgási sebessége. (int)
- **xPos**, **yPos**: A hős pozíciója a pályán. (float)

Metódusok:

- **move()**: A hős mozgásának vezérlésére.
- **jump()**: A hős ugrásának kezelése.
- **attack()**: A hős támadási mechanizmusának kezelése.
- **takeDamage()**: Ha a hős sérül, csökkenti az életerőt.
- **Heal()**: Életerő gyógyítása.
- **interact()**: Az interakciók (pl. kapcsolók, tárgyak) kezelése.

4. Enemy osztály

Az ellenségek osztálya tartalmazza azokat a jellemzőket, amelyek szükségesek a harcban való részvételhez.

Attribútumok:

- **id**: Az ellenség azonosítója. (int)

- **name**: Az ellenség neve. (string)
- **health**: Az ellenség életereje. (int)
- **damage**: Az ellenség által okozott sebzés. (int)
- **speed**: Az ellenség mozgási sebessége. (int)
- **attackRange**: Az ellenség támadási távolsága. (int)

Metódusok:

- **move()**: Az ellenség mozgásának kezelése.
- **attack()**: Az ellenség támadása.
- **takeDamage()**: Az ellenség életerejének csökkentése, ha sebzést kap.
- **patrol()**: Ha az ellenségnek van egy őrjáratozó viselkedése, akkor ezt itt vezérelhetjük.

5. Boss osztály

A főellenség osztálya, amely a legnagyobb kihívást jelenti a játékos számára, és összetettebb viselkedést mutat.

Attribútumok:

- **id**: A főellenség azonosítója. (int)
- **name**: A főellenség neve. (string)
- **health**: A főellenség életereje. (int)
- **damage**: A főellenség sebzése. (int)
- **speed**: A főellenség mozgási sebessége. (int)
- **attackPattern**: A főellenség támadási mintája (pl. véletlenszerű támadások, szakaszos támadások). (string)
- **phase**: A főellenség fázisa, ahogy a harc előrehalad. (int)

Metódusok:

- **move()**: A főellenség mozgásának vezérlése.
- **attack()**: A főellenség támadásának vezérlése.
- **takeDamage()**: Ha a főellenség sérül, csökkenti az életerőt.
- **specialAttack()**: A főellenség különleges támadása.
- **changePhase()**: A harc során változtatja a fázisait, új támadásokat és viselkedéseket vezethet be.

6. Obstacle osztály

Az akadályok osztálya azokat a fizikai akadályokat tartalmazza, amelyeket a játékosnak el kell kerülni.

Attribútumok:

- **id**: Az akadály azonosítója. (int)

- **name**: Az akadály neve. (string)
- **type**: Az akadály típusa (pl. fal, csapda, lyuk). (string)
- **xPos**, **yPos**: Az akadály pozíciója. (float)
- **width**, **height**: Az akadály szélessége és magassága. (float)

Metódusok:

- **collide()**: Amikor a hős vagy egy másik objektum ütközik az akadállyal.

7. Room osztály

A szobák osztálya a pályák különböző szintjeit és azok összes tartalmát kezeli, beleértve az akadályokat, ellenségeket és egyéb elemzőket.

Attribútumok:

- **id**: A szoba azonosítója. (int)
- **name**: Az szoba neve. (string)
- **obstacles**: Az akadályok listája ebben a szobában. (list<obstacles>)
- **enemies**: Az ellenségek listája, akik itt tartózkodnak. (list<enemies>)
- **isBossRoom**: Boolean érték, amely meghatározza, hogy a szoba a főellenség szobája-e. (bool)
- **roomType**: A szoba típusa (pl. "standard", "boss", "trap"). (string)

Metódusok:

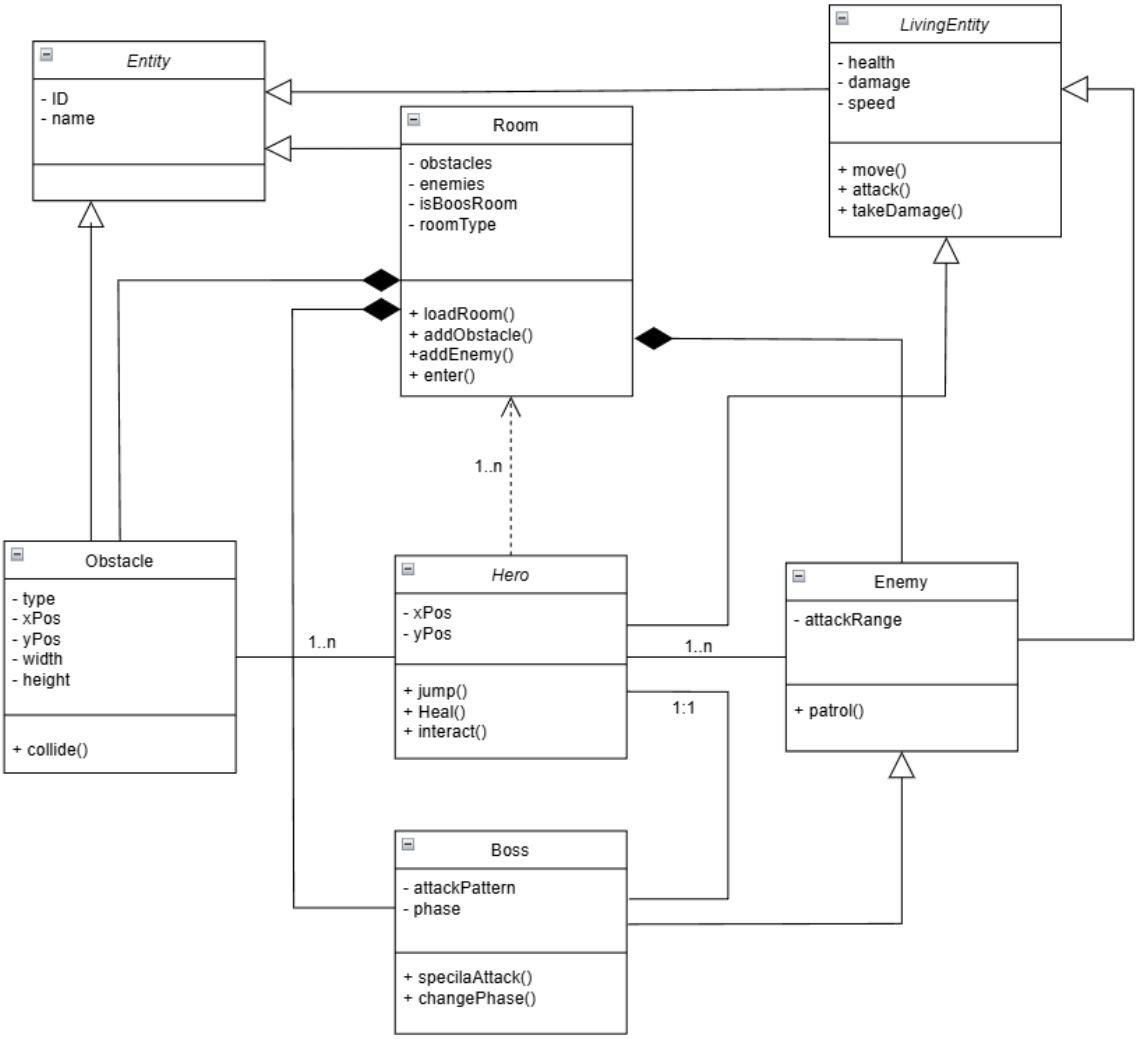
- **loadRoom()**: A szoba betöltése, amely beállítja az összes ellenséget és akadályt.
- **addObstacle()**: Új akadály hozzáadása a szobához.
- **addEnemy()**: Új ellenség hozzáadása.
- **enter()**: A szoba belépési eseménye, amely például elindítja a harcot vagy egyéb eseményeket.

Általános kapcsolat a különböző osztályok között:

- **Hero osztály** és **Enemy osztály**: A hős és az ellenségek harcolnak egymással. Az ellenségek gyakran támadnak a hősre, és a hős viszont válaszolhat a támadásaikra.
- **Hero osztály** és **Obstacles osztály**: A hős ütközhet akadályokkal, amelyeket el kell kerülnie vagy meg kell semmisítenie.
- **Boss osztály** és **Enemy osztály**: A főellenség gyakran egy speciális ellenség, amely különbözik a sima ellenségektől a támadási mintáival és erejével.
- **Rooms osztály** és más osztályok: A szobák tárolják az akadályokat, ellenségeket, és vezérlik a szobák közötti átjárásokat.

Diagram összefoglalása:

1. **Association**:
    - **Hero - Enemy**: A Hero és az Enemy közvetlen kapcsolata.
    - **Hero - Obstacle**: A Hero és az Obstacle közvetlen kapcsolata.
    - **Hero - Boss:** A Hero és a Boss közvetlen kapcsolata.
2. **Dependency**:
    - **Hero - Room**: A hős függ a szobák tartalmától és az azokban zajló eseményektől.
3. **Composition**:
    - **Enemy - Room**: Mivel az ellenségek kizárólag a szobában találhatók, és azok nem létezhetnek a szoba nélkül (tehát ha a szoba eltűnik, az ellenségek is eltűnnek), **Composition** kapcsolatot alkalmazunk.
    - **Obstacle - Room**: Az akadályok is a szobához tartoznak, és azok létezése a szoba létezéséhez van kötve. Ha a szoba megszűnik, az akadályok is eltűnnek, tehát szoros **Composition** kapcsolat van köztük.



5. ábra: Tervezett osztálydiagram

- **Boss - Room**: A főellenség is a szobában van, és szorosan összefonódik a szobával, mivel a boss csak abban a szobában létezik. Ha a szoba megszűnik, a főellenség is eltűnik, így itt is **Composition** kapcsolatot kell alkalmazni.

4. **Inheritance**:
    - **LivingEntity - Entity:** A LivingEntity absztrakt osztály az Entity absztrakt osztályból származik (öröklés).
    - **Enemy - LivingEntity:** Az Enemy osztály a LivingEntity absztrakt osztályból származik (öröklés).
    - **Boss - Enemy**: A Boss osztály az Enemy osztályból származik (öröklés).
    - **Hero - LivingEntity:** A Hero a LivingEntity absztrakt osztályból származik (öröklés).
    - **Obstacle - Entity:** Az Obstacle osztály az Entity absztrakt osztályból származik (öröklés).
    - **Room - Entity:** A Room osztály az Entity absztrakt osztályból származik (öröklés).

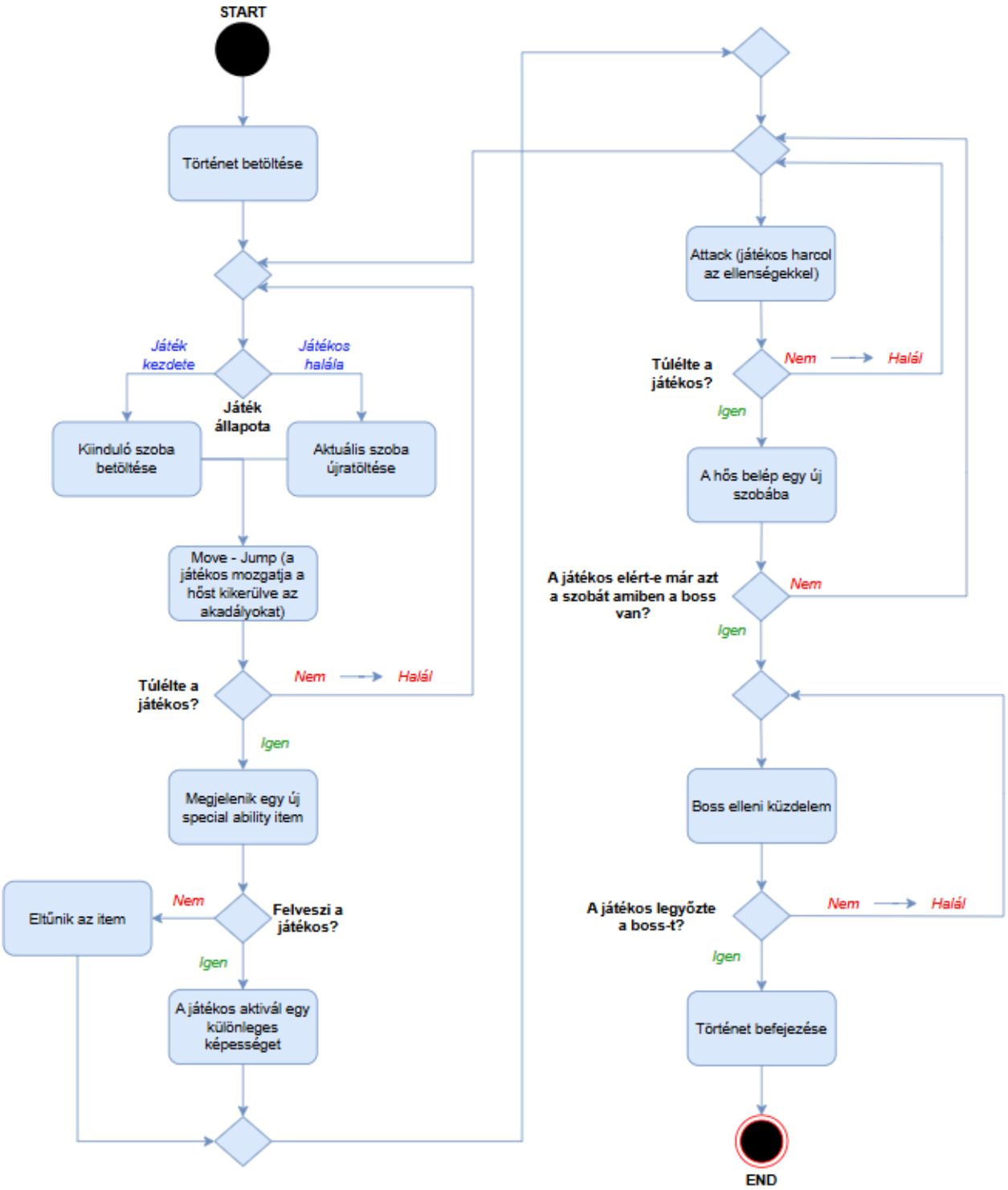
3.5. Aktivitásdiagram, viselkedés leírása

A játék általános működésének az aktivitásdiagramja a játék folyamatorientált bemutatására szolgál (**6. ábra**). Segítségével nyomon követhető, hogyan halad végig a játékos a különböző állapotokon és játékelemeken a játék során, a kezdőlépésektől egészen a játék befejezéséig. A diagram jól szemlélteti a döntési pontokat, ismétlődő folyamatokat és a játékmenet logikai sorrendjét, ezáltal hasznos eszköz a játékrendszer viselkedésének megértéséhez és a fejlesztési lépések tervezéséhez.

3.5.1. Aktivitás diagram leírása:

A program elindítása után a játékos bekerül az első kiinduló szobába. Itt a játékosnak lehetősége van saját döntésének megfelelően mozognia, ugrania vagy akár támadnia. A játék során a cél az, hogy a főhős eljusson a különböző szobákon keresztül egy végső terembe, ahol egy final boss és egy izgalmas küzdelem várja. A szobákban változatos megmérettetések várnak a játékosra, amelyek során a játékos feladata, hogy átverekjedje magát az akadályokon, illetve az ellenséges entitásokon. A szobákban megjelenhetnek különleges, a játékost segítő képességet adó objektumok, amelyek felvétele után aktiválódik az adott képesség. Amennyiben a játékos ezeket nem veszi fel az aktuális szobában, akkor eltünnek.A játék során bizonyos szakaszonként checkpointok lesznek elérhetőek. Ha játékos akár az akadályok által okozott sebzésben, akár az ellenségek támadásában meghal, akkor visszakerül az utolsó elért checkpointhoz és onnan kezdheti újra a játékot. Ez egészen addig így folytatódik, amíg a járékos el nem jut az utolsó

szobába, ahol a játék legnehezebb feladata vár rá, a boss legyőzése. Amennyiben ezt a próbát is sikeresen teljesíti, akkor a játék véget ér.



6. ábra: A játék általános aktivitásdiagramja

3.6. Boss AI logika tervezés, különböző megoldások megvizsgálása

A boss harc tervezésekor az egyik fő elérendő célom az volt, hogy a főellenfél ne csak szkriptelt minták alapján működjön, hanem reagáljon a játékos viselkedésére, és dinamikusan változtassa a taktikáját. Egy statikus, ismétlődő mintákat használó ellenfél gyorsan kiismerhető, így a küzdelem elveszíti az izgalmát és egy idő után monotonná válhat. A fejlesztés kezdetén ezért több különböző mesterséges intelligencia-szerű megközelítést is kipróbáltam, és ezen tapasztalatok alapján fokozatosan alakult ki a végleges rendszer.

3.6.1. Első verzió: Fuzzy pontozás és adaptív memória

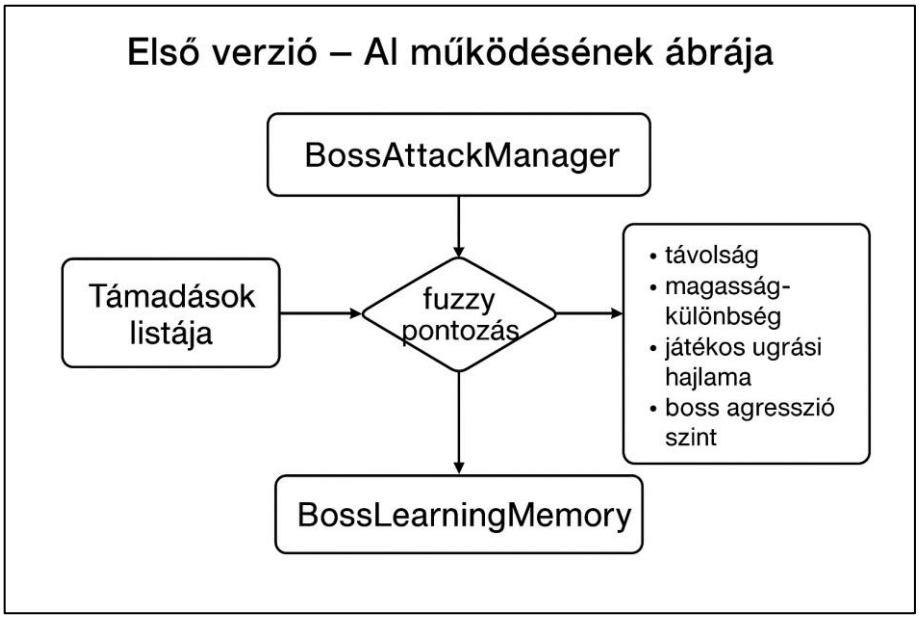
Az első verzió egy viszonylag komplex, több komponensből felépülő rendszer volt (**7. ábra**). A központi elem a *BossAttackManager* komponens, amely döntött az aktuális támadások sorrendjéről és típusáról. A döntéshozás egy egyszerűsített fuzzy logikával valósult meg, mindegyik támadás egy különböző paraméterek által befolyásolt többtényezős pontszámot kapott. Ilyen tényező volt például a játékos és a boss közti távolság, a magasságkülönbség, a játékos ugrási hajlama, a boss aktuális agressziós szintje és a korábbi támadások sikeressége. Ezeket az értékeket folytonos skálákon kezeltem, nem binárisan, így a döntések nem „igen-nem" logikával, hanem fokozatos súlyozással születtek meg.

Ahogy azt már említettem, a támadások közti választást a rendszer fuzzy pontozással végezte, amely úgy működött, hogy minden támadás kapott egy alap *score* értéket, majd ezt több szorzó módosította, például az alkalmazandó taktika típusa (agresszív, védekező, távolsági, kitérő) vagy mondjuk a tanuló memória által monitorozott hatékonyság. Ezzel elértem, hogy a boss ne mindig ugyanazt a támadást válassza az adott helyzetben, hanem az aktuális körülményekhez és korábbi tapasztalataihoz igazodjon.

A *BossLearningMemory* komponenst egy egyszerű, de hatékony statisztikai memóriaként hoztam létre. Minden támadásról nyilvántartotta, hogy hányszor használták és hányszor volt találat. Ebből egy hatékonysági mutató (*effectiveness*) számolódott, amit a döntéshozó logika a későbbiekben a súlyozáshoz használt fel. Ha egy támadás gyakran talált, annak súlya nőtt, amennyiban nem volt eredményes, akkor csökkent. Ez a rendszer nem valódi gépi tanulásként működött, de képes volt fokozatosan alkalmazkodni a játékos stílusához. A boss „emlékezett" arra, mi működött jól és azokat újra előnyben részasítette.

Az első verzióval tehát egy adaptív, statisztikus és fuzzy pontozásra épülő AI-t valósítottam meg. A tesztek alapján a viselkedés meglepően természetesnek bizonyult, a boss reagált a játékos taktikáira és egyre hatékonyabb lett az idő haladtával. Ugyanakkor a rendszer bonyolultsága miatt nehezebb volt karbantartani, a feltételes logika gyorsan túlburjánzott, és a

döntések nehezen követhetők lettek.



7. ábra: Első AI működés logika

Komponensek:

1. **BossAttackManager**: központi döntéshozó komponens
2. **Támadások listája**: minden lehetséges támadás, rendelkeznek alap *score*-ral
3. **Fuzzy pontozás** (többtényezős súlyozás)
   - *távolság a player és a boss között*
   - *magasságkülönbség a player és a boss között*
   - *játékos ugrási hajlama*
   - *boss agressziójának szintje*
   - *korábbi támadások sikeressége*
4. **BossLearningMemory**: statisztikus memória
   - *minden támadás sikerességének rögzítése*
   - *súlyozás a döntéshez*

Adatáramlás:

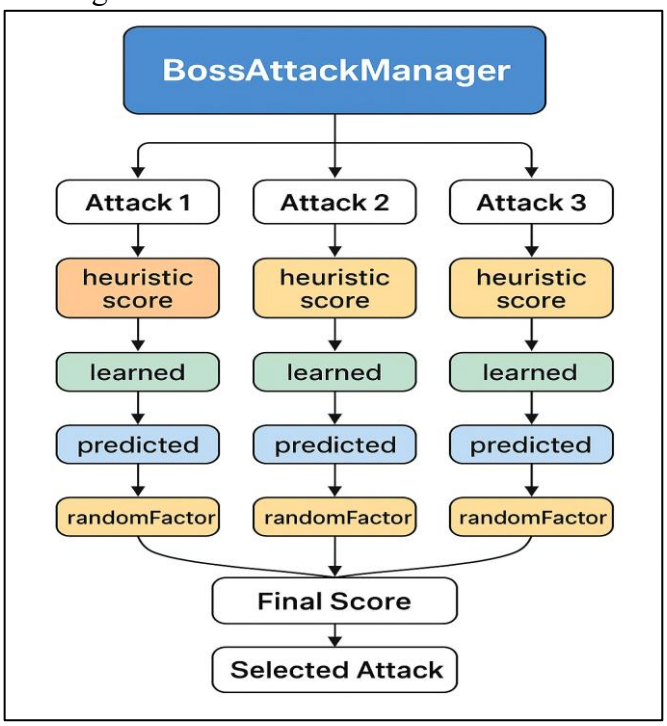
- *BossAttackManager* lekéri a támadások aktuális állapotát a fuzzy pontozásból
- A *BossLearningMemory* frissíti a súlyokat minden támadás után
- Fuzzy pontozás + memória = *BossAttackManager* kiválasztja a végrehajtandó támadást

Ábra jelölések:

- *Téglalap* = komponens
- *Nyíl* = adat/utasítás áramlása
- *Döntési pont* = fuzzy score számítás

3.6.2. Második verzió: Prediktív, heurisztikus döntéshozás

A második verzió fejlesztésekor az volt a gondolatom, hogy egy egyszerűbb, de előrelátóbb döntési logikát hozzak létre (**8. ábra**). A korábbi fuzzy állapotgépet lecseréltem egy modulárisabb, pontozásos megoldásra, ahol minden támadás önállóan számította a saját heurisztikus értékét. A fő komponens továbbra is a *BossAttackManager* maradt, de a döntéshozás logikája némileg átalakult.



8. ábra: Második AI működés logika

A döntés ebben az esetben négy fő tényező összegeként állt elő:

- **heuristicScore:** az adott támadás saját heurisztikája (például távolság, sebzési potenciál, mozgásidő),
- **learned:** a memória alapján mért hatékonysági korrekció,
- **predicted:** a játékos várható reakciójának becslése,
- **randomFactor:** kiszámíthatatlanságot biztosító véletlen elem.

A legfontosabb újítás a prediktív komponens volt. Készítettem egy *PredictPlayerReaction()* függvényt, amely megfigyelte a játékos ugrási hajlamát, távolságát, agresszióját, és ezekből következtetett a várható mozdulatra. Ha például a játékos gyakran ugrott, a rendszer előnyben részesítette a felfelé irányuló támadásokat; ha inkább közelharcra törekedett, a boss több dash-jellegű támadást használt. Ez a megoldás kvázi „előre gondolkodó” viselkedést adott a főellenfélnek, ami emberibbnek hatott játék közben.

A második verzió könnyebben karbantartható volt, és a döntési logika letisztultabb lett, ugyanakkor a fuzzy logika kikerült a rendszerből, így a viselkedés néha túl kiszámíthatónak bizonyult. A taktikai váltások és a „hangulatváltás” (pl. agresszívből védekező mód) hiánya miatt a boss kevésbé reagált a körülményekhez hűen. Bár az AI heurisztikus és prediktív lett, a tesztek alapján a viselkedés bizonyos helyzetekben mechanikusnak érződött. A memóriakomponens továbbra is működött, de a hatása gyengébb, elenyészőbb lett, mivel a statisztikai visszacsatolás kisebb súlyt kapott a végső pontszámításban.

Fő komponensek:

1. **BossAttackManager**: a fő döntéshozó
2. **Támadások**: mindegyik számítja a saját heurisztikus értékét
3. **Döntési tényezők**

Az ábra mutatja, hogy minden támadás külön értékeli magát a négy tényező alapján, majd a *BossAttackManager* kiválasztja a legmagasabb pontszámú támadást.

3.6.3. Harmadik verzió: Kibővített fuzzy rendszer és adaptív komponensek

A két korábbi megközelítés tapasztalatai alapján a végleges irány meghatározásában a választásom egy kibővített fuzzy logikán alapuló rendszerre esett (**9. ábra**). Ez ötvözte az első verzió adaptív, kontextusfüggő döntéseit a második verzió modularitásával és tisztább szerkezetével. Ebben a rendszerben a fuzzy logika már nemcsak a támadások kiválasztásában, hanem több komponensben is jelen van. A *BossAttackManager* a taktikai döntéseket fuzzy szabályokkal határozza meg, a *BossTeleport* fuzzy logikát használ a teleportálási célhely meghatározásához, míg a *BossRoomController* a health és invulnerability pickup item-ek spawn esélyeit számítja fuzzy értékek alapján.

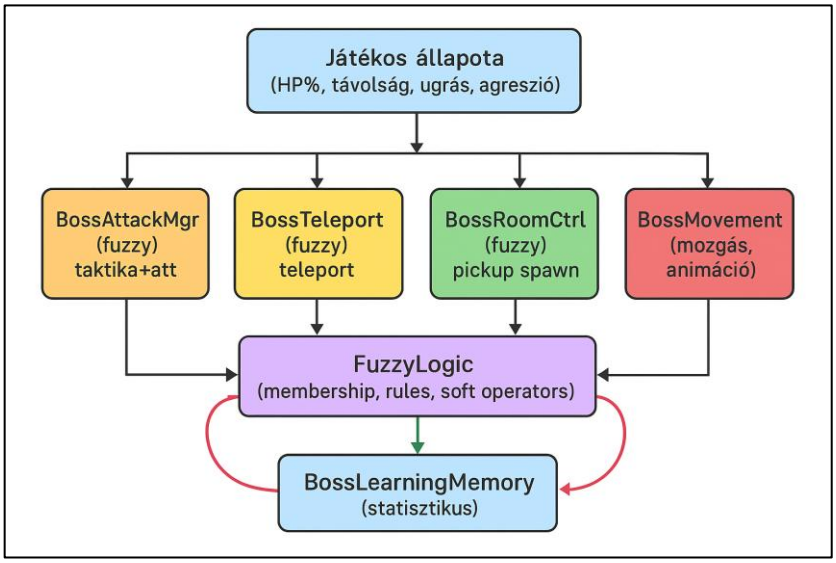
A fuzzy logika előnye, hogy nem merev határokkal dolgozik. A változók (pl. HP, agresszió, távolság) 0 és 1 közötti tartományban kapnak fuzzy értékeket, amelyek fokozatosan változnak. Ahogy már az első verziónál is utaltam rá, ez lehetővé tette, hogy a boss döntései ne binárisak, hanem „folytonosak” legyenek, a taktikai váltás például nem hirtelen, hanem fokozatosan történik meg, a játékos viselkedésétől függően. Ezen felül a *BossLearningMemory* továbbra is megtartotta az önkorrekciós mechanizmust, így a boss emlékszik a korábbi sikereire, és azokat beépíti a fuzzy pontozásba és ezáltal a döntésbe.

Ez a verzió adta a legkiegyensúlyozottabb kompromisszumot:

- a fuzzy logika miatt a viselkedés emberibb és kiszámíthatatlanabb,
- a memória miatt adaptív,

- a modularitás miatt könnyen bővíthető lett.

A tesztek során ez a megoldás hozta a legstabilabb eredményt. A boss természetesen reagált a játékos stílusára, a harc dinamikája folyamatosan változott, és a nehézségi szint harmonikusan igazodott a játékos teljesítményéhez. A fuzzy szabályokkal kombinált statisztikus memória így egy olyan „AI-szerű” viselkedést eredményezett, ami gépi tanulás nélkül is intelligensnek érződött.



9. ábra: Harmadik AI működés logika

Magyarázat az ábrához:

- A **játékos állapota** a fő input minden fuzzy döntéshez.
- **BossAttackManager**, **BossTeleport**, **BossRoomController** mind fuzzy logikát használ.
- **BossMovement** aktiválja a többi komponenst, de nem használ fuzzy logikát.
- **BossLearningMemory** visszacsatol adatot minden fuzzy döntéshez: agresszió, hatékonyság, ugrásstílus.
- A **FuzzyLogic** központi könyvtár, amit a modulok hívnak a számításokhoz.

Az ábra kiemeli a **visszacsatolás** mechanizmust, ami adaptívvá teszi a boss viselkedését.

3.6.4. Összegzés

Összegzésként elmondható, hogy a fejlesztés során több különböző irányt próbáltam ki. Az első verzió komplex, adaptív, de nehezen kezelhető volt. A második letisztultnak, de kevésbé változatosnak bizonyult. A harmadik pedig egyesítette a kettő előnyeit és az eredmény egy stabil, több komponensre épülő, fuzzy logikát és statisztikus tanulást kombináló rendszer lett, amely képes reagálni a játékos viselkedésére és fokozatosan módosítani tudja a saját taktikáját.

4. Implementáció, megvalósítás

4.1. Alapvető játékmechanika

4.1.1. Player logika

A terv szerinti *Hero* osztály helyett, ami a LivingEntity absztrakt osztályból származott volna le és egyben tartalmazta volna az összes funkciót és attribútumot, egy több scriptből álló rendszert alakítottam ki. Eszerint egymástól elkülönülő komponensek felelősek minden fontosabb funkcióért. Így egyszerűbben, egymástól elkülönülve tudtam kezelni a mozgásokat, a támadást és az újraéledés logikáját is. Igyekeztem úgy kialakítani a scripteket, hogy az egyes részek között ne legyen felesleges átfedés, ezzel a hibakeresést és a karbantarthatóságot is megkönnyítettem.

Ahogy az a nevéből is kikövetkeztethető, a *PlayerMovement* komponens tartalmazza a karakter minden mozgási logikáját. Ide tartoznak a vízszintes mozgások, a guggolás és az ugrások, amelyeket egyszerű input-velocity logika vezérel, valamint például a falra mászás is (**10. ábra**), amelyet pedig a raycast alapú talaj és falérzékelés szolgáltat, hogy a karakter helyesen tudjon reagálni a környezetére (**1. kódrészlet**).

10. ábra: Falra mászás

```
public bool CheckGround()
{
    RaycastHit2D raycastHit = Physics2D.BoxCast(boxCollider.bounds.center,
                                boxCollider.bounds.size, 0f, Vector2.down, 0.1f,
groundLayer);
    return raycastHit.collider != null;
}

private bool CheckWall()
{
```

```
        RaycastHit2D raycastHit = Physics2D.BoxCast(boxCollider.bounds.center,
                                    boxCollider.bounds.size, 0f, new
                                    Vector2(transform.localScale.x, 0), 0.1f, wallLayer);
        return raycastHit.collider != null;
}
```

1. kódrészlet: Raycast alapú érzékelés

Mivel a platformer játékok alapvető és talán legfontosabb mechanikája az ugrás, ki szeretném emelni, hogy ez a játékomban miként működik (**2. kódrészlet**). Egy olyan ugrás logikát alakítottam ki, amelyben az ugrás magassága függ gomblenyomás hosszúságától, lehetőség van dupla ugrás végrehajtására és „coyote time" mechanika is segíti a játékosokat. Ez lehetővé teszi, hogy a karakter akkor is ugorhasson, amikor már elhagyta a talajt vagy az adott platformot, de csak egy rövid ideig. Ez felel azért, hogy a dupla ugrás végbe mehessen, valamint ennek köszönhető az is, hogy amikor a játékos mondjuk véletlenül leesik egy akadályról akkor a „coyote time"-nak megfelelő ideig még ugorhat a levegőben is.

A *PlayerAttack* script kezeli a főhős támadását. Ez igazából jelenleg egy egyszerű logika alapján működik. A karakter rendelkezik egy *fire* ponttal, ami a player pozíciójának és irányának megfelelően folyamatosan változik a karakter helyzetével együtt, de a karakterhez képest fix. Társul ehhez a támadáshoz természetesen egy animáció, illetve egy hanghatás is ezzel növelve a játékélményt (**11. ábra**). A lövedékek újrahasznosítási rendszere „projectile pooling"-gal valósul meg. Ez optimalizálja a játék teljesítményét, hiszen lövéskor nem minden esetben egy új objektumot kell létrehozni, hanem előre definiált példányokat aktivál és deaktivál a játék.

11. ábra: Player támadása

A lövedékek is rendelkeznek egy saját scripttel, ez pedig a *Projectile* komponens. Ebben igazából a *CompareTag*-ek vannak kezelve, vagyis, hogy a különböző objektumokkal történő ütközésnél mi történjen. Amennyiben valamilyen ellenséggel ütközik a lövedék akkor csökkentjük az ellenség életerejét, viszont, ha például valami item-mel vagy checkpoint objektummal ütköznek a Collider-ek, akkor az simán tovább engedi a lövedéket.

A *PlayerRespawn* komponens kezeli a játékos halála utáni újraéledést. Ezen játékmechanikának

jelentős szerepe van abban, hogy a játék ne legyen túlzottan bűntető jellegű, ugyanakkor megőrizz a rage game-ek által nyújtott kihívásokat is. Ennek a rendszernek az alapvető elemei a checkpointok, amelyek előre meghatározott helyeken helyezkednek el és igyekeznek segítséget nyújtani a játékosnak (**12-13. ábra**).

12. ábra: A checkpoint felvétel előtt

13. ábra: A checkpoint felvétel után

Ameddig a játékos nem vett fel checkpointot, addig kizárólag az elejéről kezdheti újra a játékot, azonban miután aktivált már checkpointot, az ő döntése lesz, hogy halál után az elejéről szeretné újra kezdeni a játékmenetet vagy a legutolsó elért checkpointtól. Ha úgy dönt azonban, hogy az elejéről kezdi újra, akkor az addig megszerzett mentett pozíciók elvesznek.

Amennyiben az elejéről kezdi újra a játékot, akkor minden a kezdeti állapotba kerül. Viszont, ha egy checkpointtól indul újra, akkor a pontszám visszaáll arra az értékre, amivel rendelkezett a játékos, amikor felvette a checkpointot. Az életerő viszont minden esetben teljesen visszatöltődik újraéledéskor, a kamera pedig arra a szobára ugrik vissza, ahova kerül a játékos (**3. kódrészlet**).

```
public void LoadFromCheckpoint()
{
    if (currentCheckpoint != null || checkpointPosition != Vector3.zero)
    {
        //Pozíció visszaállítása
        Vector3 respawnPos = currentCheckpoint != null ? currentCheckpoint.position :
                    checkpointPosition;
        transform.position = respawnPos;

        //Élet visszaállítása
        playerHealth.Respawn();

        //UI visszaállítása
        uiManager.gameOverScreen.SetActive(false);
        Time.timeScale = 1;
```

```
            //Pontszám visszaállítása
            if (GameScoreManager.Instance != null)
            {
                GameScoreManager.Instance.SetScore(GameScoreManager.checkpointScore);
            }
            //Kamera visszaállítás logika
        }
        else
        {
            // Ha nincs mentett checkpoint
            SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
            Time.timeScale = 1;
        }
}
```

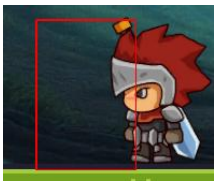
3. kódrészlet: LoadFromCheckpoint metódus részlete

A játékos életerejét egy központilag használt *Health* komponens kezeli. Itt egy *currentHealth* változóval folyamatosan nyomon van követve az életerő. Amennyiben ez nulla, akkor itt található az a logika, ami kezeli a player halálát. Ezt a *TakeDamage()* a *Die()* és a *Respawn()* metódusok valósítják meg (**4. Kódrészlet**). A *TakeDamage()* figyeli, hogy a *currentHealth* nulla-e és amennyiben annyi, akkor meghívja a *Die()* függvényt. Az újraéledésről pedig a *Respawn()* metódus gondoskodik.

4.1.2. Ellenségek logikája

Az ellenségek esetében a terv szerint szintén a *LivingEntity* absztrakt osztály lett volna az alap. Ebből származott volna az *Enemy* osztály, majd ebből öröklődött volna tovább a Boss osztály is. Mivel az alap ellenségek és a boss mechanikája jelentősen eltér egymástól, ezért egyszerűbbnek és logikusabbnak tűnt, ha egymástól elkülönülve alakítom ki őket. A játékom jelenlegi verziójában kétfajta ellenséget hoztam létre. Az egyik a közel harci ellenség, a játékban MeleeEnemy (**14. ábra**), a másik pedig egy távolsági ellenség, a játékban RangedEnemy (**15. ábra**). Mindkettő ellenség Collider alapú detektációval működik.

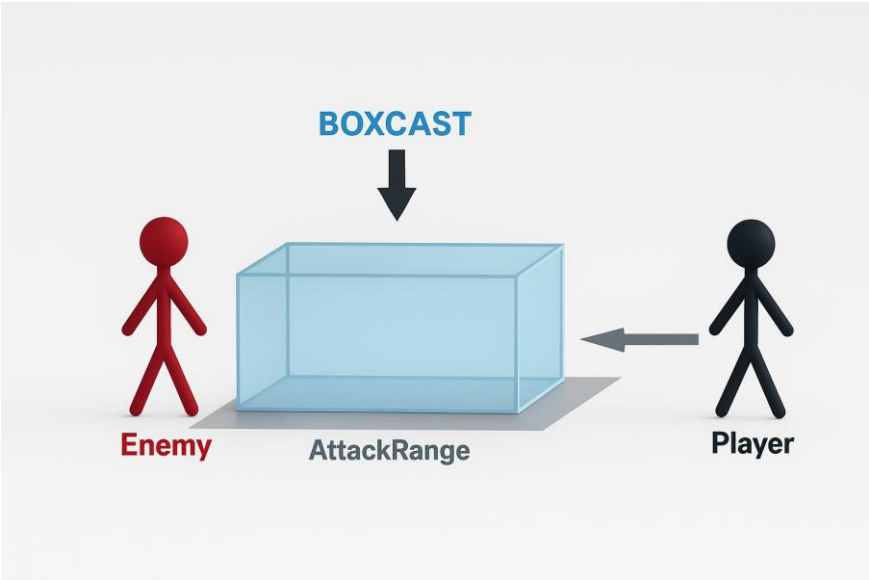
14. ábra: RangedEnemy



15. ábra: MeleeEnemy

A közelharci ellenségek *BoxCast* használatával viszonylag közelről érzékelik csak a játékost. A BoxCast lényegében egy doboz alakú „láthatatlan érzékelő”, amelyet a Unity előre bocsát egy irányba, és megmondja, hogy ütközik-e valamivel (például a játékossal).Ez főként közelharci támadásoknál használatos, megnézi, hogy a játékos a támadási tartományon belül van-e (**16.**

ábra). Az erre használt metódusom a *PlayerInSight()*, ami visszaad egy logikai értéket, amelyet az enemy *Update()* metódusa figyel és amint ez igaz lesz, az animator triggerje is igazra vált és végrehajtódik a támadás (**5. kódrészlet**). Tehát amikor a játékos belép a MeleeEnemy látótávolságába, akkor indítja meg az ellenség a támadását, ami egy kardcsapás formájában valósul meg. Ekkor az animációval egy *Event*-ként hívódik meg a *DamagePlayer()* függvény (**6. kódrészlet**).



16. ábra: Enemy BoxCast

```
private void DamagePlayer()
{
    if (playerHealth != null && PlayerInSight() && playerHealth.CanGetDamage())
        playerHealth.TakeDamage(attackDamage);
}
```

6. kódrészlet: MeleeEnemy DamagePlayer metódusa

A távolsági ellenségek hasonló logikai alapon csak nagyobb észlelési zónával működnek. Amikor a player belép a RangedEnemy Támadási zónájába, akkor az enemy tűzgolyót lő rá. A lövedékek itt is egy előre létrehozott pool-ból kerülnek kiválasztásra, ahogy az a player-nél is működik. Hasonló módon működik a mechanika, mint a MeleeEnemy esetében, az *Update()* metódus itt is folyamatosan figyeli, hogy a player belépett-e az észlelési zónába. Amikor ez megtörténik, akkor itt is egy animator *Event*-tel hívódik meg a *RangedAttack()* függvényem (**7. kódrészlet**).

```
private void RangedAttack()
{
    SoundManager.instance.PlaySound(fireballSound);
```

```
        cooldownTimer = 0f;

        GameObject projectile = projectiles[FindInactiveProjectile()];
        projectile.transform.position = firepoint.position;
        projectile.GetComponent<EnemyProjectile>().ActivateProjectile();
}
```

7. kódrészlet: RangedEnemy RangedAttack metódusa

Az ellenségek alapvetően statikusan állnak egy pozíción és várnak a player-re, de létrehoztam egy *EnemyPatrol* komponenst, amivel viszont két megadott pont között mozognak és járőröznek. Erre szabadon ráhúzható bármelyik enemy. A két pont közötti irányváltás nem azonnal történik, amikor az ellenség eléri az egyik szélét a mozgásnak, akkor csak egy bizonyos idő eltelt után fordul vissza. Ezzel egy kicsit természetesebbnek hat a mozgás. Itt igazából csak egy egyszerű mozgás mechanikát alkalmaztam, ezért nem gondolom, hogy ezt érdekes lenne itt megjeleníteni.

Mindkét ellenség típus rendelkezik a központi *Health* komponenssel és az életerejük csökkenése hasonlóan működik, mint ahogy azt a player-nél már láthattuk. Az enemy-k újraélesztésénél pedig a *Health* komponens *ResetHealth()* függvényét hívja meg az enemy *ResetEnemy()* metódusa (**8. kódrészlet**).

4.1.3. Csapda logikák

A terv szerint az akadályokat is egy osztály használatával alakítottam volna ki, azonban ez a fejlesztés során problémás megoldásnak bizonyult. Ebben a központi Obstacle komponensben kellett volna kezelni az összes akadály logikáját és ez a hibakeresés és karbantartás szempontjából nagyon nehézkes kialakítást eredményezett volna. Ezért végül minden akadályt külön MonoBehaviour-ként alakítottam ki a könnyebb kezelhetőség kedvéért.

A jelenlegi verziójú játékomban hét különböző csapdát hoztam létre. Vannak köztük egyszerűbbek és vannak összetettebb mechanikák. A csapdák logikájához is a *Colllider2D* komponenst használtam fel. A legegyszerűbb csapda a *Spikes*, ez valójában annyit csinál, hogy amikor a player hozzáér, akkor sebez rajta egyet (**17. ábra**). A *SpikeHead* csapda lényege, hogy amikor detektálja a főhős karakterét, akkor elkezd felé mozogni és addig követi, ameddig össze nem ütközik vele, ilyenkor pedig sebez a player-en (**18. ábra**). Ehhez a detektáláshoz a *ScanForPlayer()* metódust használja fel (**9. kódrészlet**).

17. ábra: Spikes

18. ábra: SpikeHead

A következő csapdám a *Fire*. Ez lényegében csak folyamatosan ég és amikor ennek neki megy a player sebez (**19. ábra**). Ennél egy fokkal érdekesebb csapda a *FireTrap*, ami úgy működik, hogy amikor a player rámegy, akkor egy trigger bekapcsolja a tüzet egy kis késleltetéssel, vagyis ennek a lényege, hogy a player ne tartózkodjon túl sokáig egyhelyben (**20. ábra**). Miután bekapcsolt a *FireTrap* csapda, akkor már ugyanúgy működik, mint a sima *Fire*.

19. ábra: Fire

20. ábra: FireTrap triggerelt állapotban

Egy másik csapda a *Saw* (**21. ábra**). Be lehet állítani, hogy ez egy megadott pozícióban legyen vagy pedig egy bizonyos területen mozogjon jobbra balra miközben működik, vagyis forog. Ez is igazából a *Collider2D* komponens segítségével érzékeli, ha a csapda ütközik a Player tag-gel rendelkező objektummal és ebben az esetben sebez egyet a főhősön. További csapda még az *ArrowTrap*, aminek a működése is egész egyszerű, hiszen miután lehelyeztük egy pozícióba, akkor bizonyos időközönként kilő egy nyilat a csapdához hozzárendelt *firepoint*-ból (**22. ábra**). A nyilak kiválasztása itt is a projectile-oknál már megszokott módon objektum pooling rendszerrel történik (**10. kódrészlet**).

21. ábra: Saw

22. ábra: ArrowTrap

A jelenlegi utolsó csapda pedig a *FallingPlatform*, ami a nevének is megfelelően olyan funkciót valósít meg, hogy amikor a player rálép vagy ráugrik, akkor egészen egyszerűen leesik. Ez annak köszönhető, hogy eredetileg a platform kinematikus, vagyis nem hat rá a gravitáció, de amikor hozzáér a player, akkor át vált dinamikusra és a gravitáció miatt lezuhan (**11.**

kódrészlet). Általában ezt a játékban egy másik csapdával együtt használtam. Például fölé helyeztem egy *Fire* csapdának. Ez a platform ugyanúgy néz ki, mint egy normál platform, hiszen ebben az a nehézég, hogy amikor a player nem számít rá, akkor egyszer csak lezuhan.

4.1.4. UI elemek logikája

Ennél a résznél bemutatnék néhány fontosabb UI elemet. A boss harc során például létrehoztam egy életerő csíkot, amin látszik, hogy a boss-nak éppen mennyi élete van (**23. ábra**). Ez folyamatosan figyeli a boss-t ért találatokat és ennek megfelelően aktualizálja az UI-t. Valójában itt egy egyszerű *fillAmount*-os megoldást használtam, amely az *Update()* metódusban folyamatosan figyeli, hogy kell-e változnia (**12. kódrészlet**). Ez leginkább a játékos érdekeit szolgálja, hiszen láthatja, hogy még mennyi sebzést kell leadnia a főellenségre.

23. ábra: Boss HealthBar

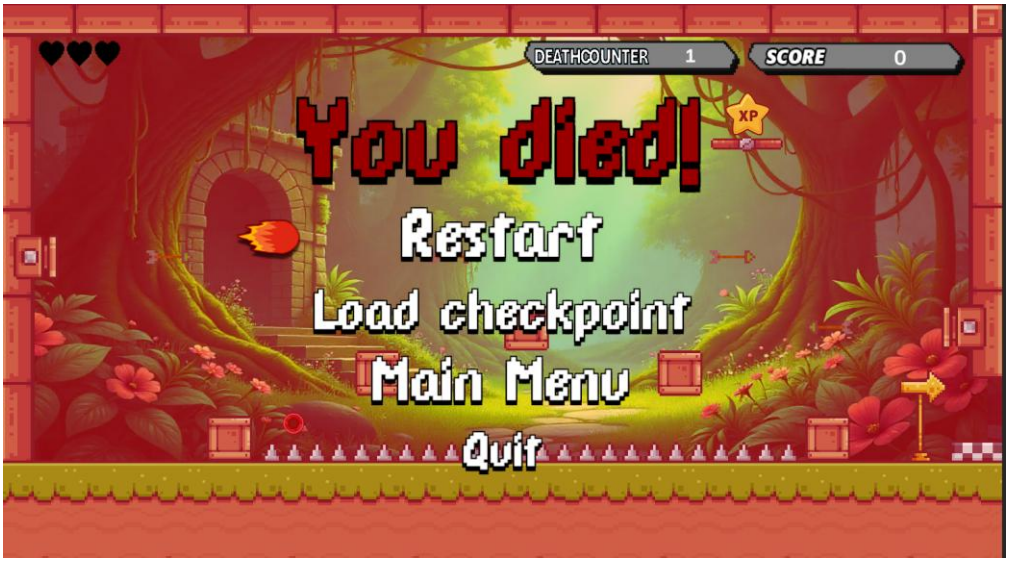
Egy másik fajta visszajelzésként a játékosnak létrehoztam egy deathcounter UI-t is (**24. ábra**). Ez igazából azon felül, hogy a játékost tájékoztatja, hogy hányszor halt már meg a játék során, kicsit azt a célt is szolgálja, hogy még nagyobb nyomást gyakoroljon a játékosokra egy jó rage game típusú játékhoz méltóan. Egy másik statisztikai UI elem a gameScore(**25. ábra**). Ez is hasonlóan visszajelzésként szolgál a játék során, illetve ennek a gyűjtögetése plusz feladatot ad a játékosoknak. Az egyes ellenségek legyőzése, a checkpointok elérése és extra Score objektumok felvételével lehet növelni a pontszámot.

24. ábra: DeathCounter UI

25. ábra: GameScore UI

A játékomban vagy egy *UIManager* komponens, ami kezeli az egyes *Screen*-ek közötti váltásokat. Az egyik legfontosabb ilyen a *GameOver Screen*, amely a főhős halála esetén jelenik meg (**26. ábra**). ezalatt a játékot megállítjuk és letiltunk minden olyan funkciót, ami nem része

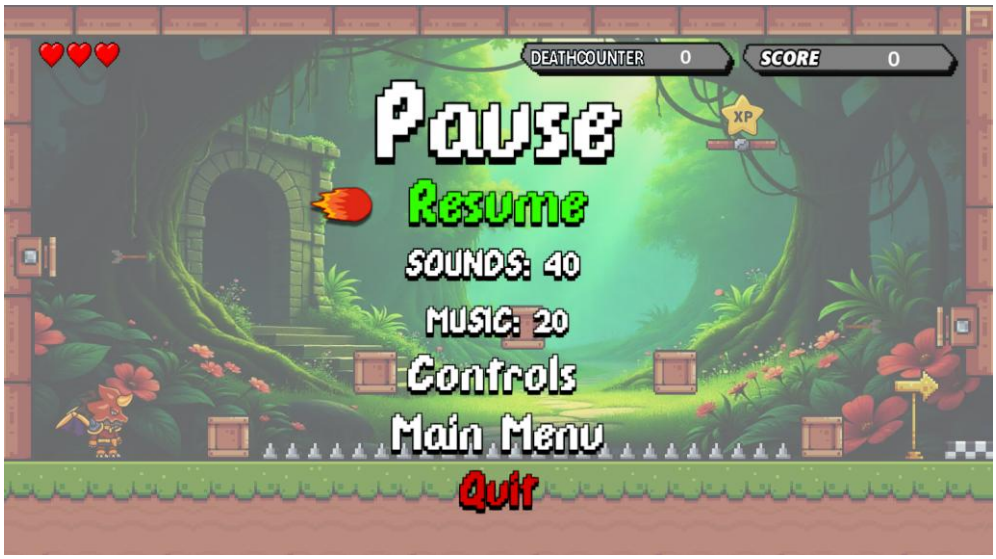
ennek a *Screen*-nek. A *GameOver Screen* aktiválásához a *Health* script nyújt segítséget, hiszen ez figyeli, hogy a játékos mikor hal meg. Ezen a képernyőn a játékos négy lehetőség közül választhat. A *Restart* gombra kattintva teljesen az elejéről kezdődik a játék, ha a *Load checkpoint* elemet választja és már van elért checkpoint, abban az esetben oda kerül vissza a játékos, ha azonban még nincs checkpoint, akkor szintén a legelejéről kezdődik a játék. Illetve van még lehetőség a *MainMenu*-be történő visszatérésre a *Main Menu* gombra kattintva, illetve a játékból való kilépésre a *Quit* lehetőséget választva.



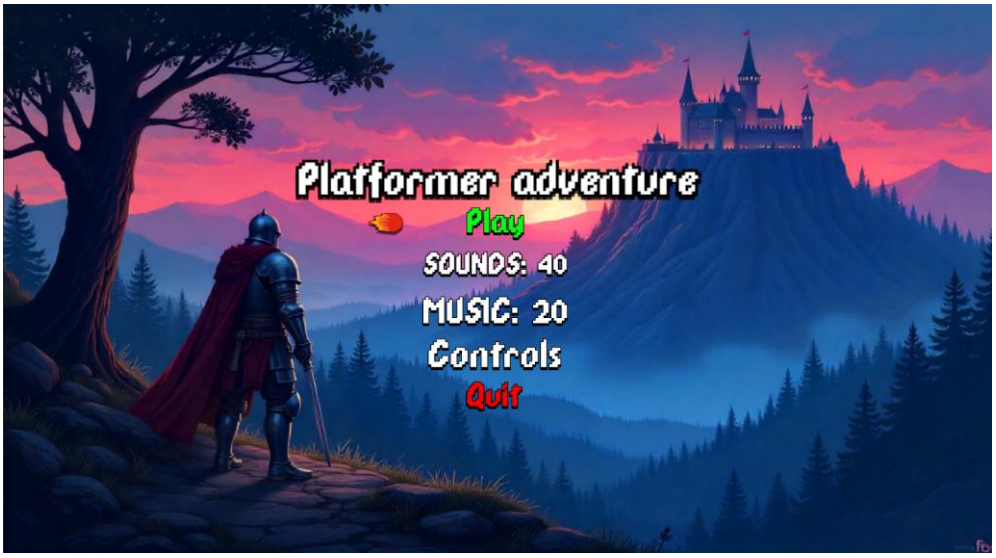
26. ábra: GameOver Screen

Egy másik előhívható Screen a *PauseScreen* (**27. ábra**). Ez az „Esc” billentyű lenyomásával aktiválható és deaktiválható egyaránt. Ezen képernyő aktiválásakor is leáll a játék. Itt a játékosnak 6 lehetősége van. A *Resume* gombbal folytathatja a játékot, a *Sounds* lehetőséggel a játék hangeffektjeinek hangerejét tudja állítani, míg a *Music* gombbal a játék zenéjét szabályozhatja. Valamint itt is lehetőség van a *Control* gomb megnyomása után a játék irányítási mechanikáinak a megtekintésére, továbbá vissza lehet lépni a *MainMenu*-be és ki is lehet lépni a játékból.

A *MainMenu* nagyon hasonlóan van felépítve ezekhez, de mivel az egy külön *MainMenu Scene*-en található, ezért saját komponenst hoztam neki létre, ez a *MainMenuManager* (**28. ábra**). A játékos itt is az ismert lehetőségek közül választhat. A *Play* gombbal elkezdheti a játékot, a *Sounds* és a *Music* lehetőségekkel a hangok szabályozhatók, míg a *Controls* választása esetén az irányítás tekinthető meg, valamint *Quit*-tel ki lehet lépni a játékból.



27. ábra: PauseScreen



28. ábra: Main Menu Screen

Ezeken a Screen-eken a gombok közötti váltáshoz, amennyiben ezt a billentyűzetről végezzük, látszik, hogy van egy *SelectionArrow*. Ez igazából csak dizájn elem a játékban. Egy listaként be vannak adva neki a lehetőségek, amik között lehet válogatni és ez alapján képes mozogni. A mozgást a *ChangePostition()* metódussal, míg a kiválasztást az *Interact()* függvénnyel valósítja meg (**13. kódrészlet**).

```
private void ChangePosition(int _change)
{
    currentPosition += _change;

     if (_change != 0)
        SoundManager.instance.PlaySound(changeSound);

    if (currentPosition < 0)
```

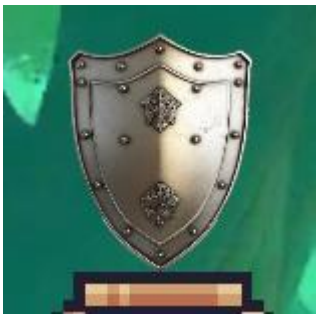
```
            currentPosition = options.Length - 1;
        else if (currentPosition > options.Length - 1)
            currentPosition = 0;

        rect.position = new Vector3(rect.position.x, options[currentPosition].position.y, 0);
}

private void Interact()
{
        SoundManager.instance.PlaySound(interactSound);

        options[currentPosition].GetComponent<Button>().onClick.Invoke();
}
```

13. kódrészlet: ChangePosition és Interact metódusok

4.1.5. Felvehető objektumok logikája

A játékélmény fokozására és a játékos segítésére létrehoztam négy különböző felvehető *item*-et. Ezen objektumok felvétele igazából a *Collider* komponens használatával történik. Van egy *HealtCollectable* objektumom, aminek a felvétele esetén a player növeli saját életerejét abban az esetben, ha éppen nem a maximális 3 életponttal rendelkezik, hanem kevesebbel (**29. ábra**). A másik felvehető *item,* ami a player életerejével kapcsolatos az az *Invulnerability* (**30. ábra**). Ennek a lényege, hogy a felvétele után a játékos egy bizonyos ideig sebezhetetlen lesz. Ehhez a *Health* komponens *StartPowerupInvulnerability()* metódusát hívjuk meg (**14. kódrészlet**). Az *isVoulnerable* változó állítgatásával szabályozzuk, hogy a *Health* komponens *TakeDamage()* függvénye levonja-e a player életerejét vagy ne, amikor sebződne.

29. ábra: HealtCollectable



30. ábra: Invulnerability

További felvehető segítség az *ExtraSpeed* is (**31. ábra**). Ahogy a nevéből is ki lehet találni, ezen objektum felvétele után a player gyorsasága és ugrásának ereje megnövekszik egy bizonyos értékkel. Ez egy viszonylag nagy segítség, ezért egyszer használtam csak a játékban, amikor a pálya teljesítése erre a segítségre épült. Az *item* felvétele után az *OnTriggerEnter2D()* metódus meghívja a *PlayerMovement* komponens *ApplySpeedBoost()* függvényét (**15. kódrészlet**). Ez a *Unity Inspector*-ból állítható paramétereknek megfelelően módosítja a főhős

mozgását.

Az utolsó, vagyis negyedik felvehető objektumom a *PlusXP* (**32. ábra**). Ha játékos felveszi ezt, akkor egész egyszerűen hozzáadódik a pontszámhoz egy bizonyos érték, jelenleg ez a játékban 300 pont. Ez a *ScoreEvents* komponens *AddScore()* metódusának meghívásával történik. Ezeket az objektumokat a játékban általában olyan helyre helyeztem el, ahol kicsit nehezebben lehet csak elérni őket, illetve plusz erőfeszítéseket igényel ezeknek a felvétele. Ezzel próbáltam kicsit differenciálni a játékosok teljesítményét.

31. ábra: ExtraSpeed

32. ábra: PlusXP

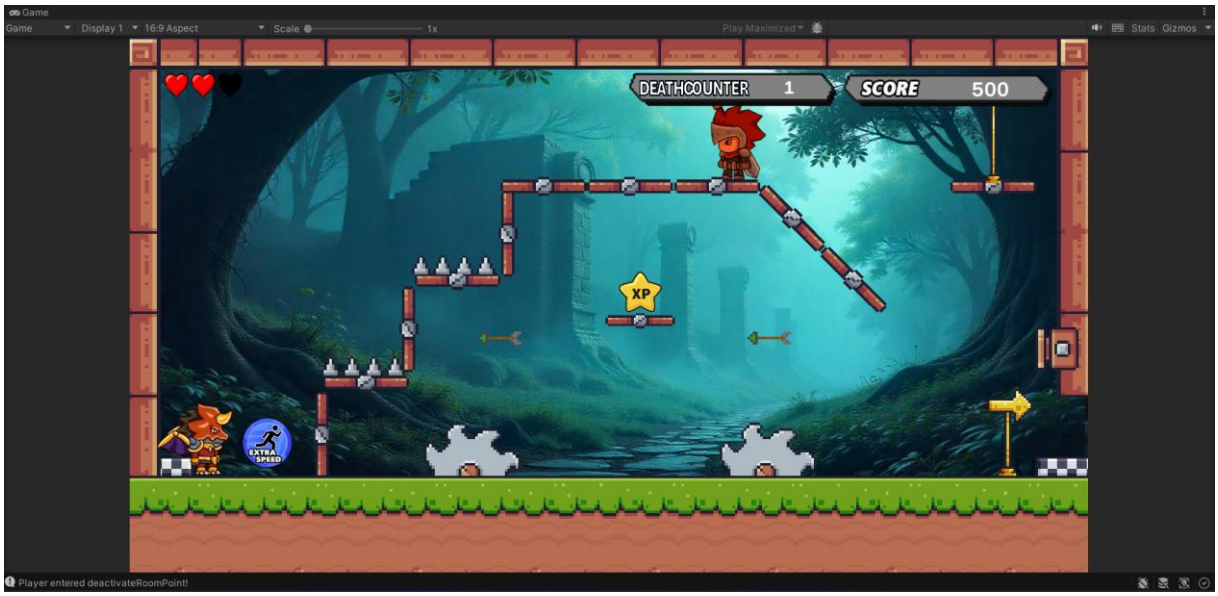
4.1.6. Kamera, szobák logikája

A kamera mozgásához kétféle megközelítést is kipróbáltam. Az első a player-t követte, nem fix pontoknak megfelelően mozgott. A másik fixen arra a szobára mozdul át amelyikben a player éppen tartózkodik. Végül ezt a fix pontos megoldást választottam. Ezt a *MoveToNewRoom()* metódusom valósítja meg (**16. kódrészlet**), amelyet akkor hívunk meg, amikor a főhős átlép a szobák *Door* objektumán, vagyis ez egy trigger-ként működik a játékban. Jelenleg a játékban minden szobában található egy *CameraPoint* és valójában erre áll rá aktuálisan mindig a kamera a játékos tartózkodásának megfelelően (**33. ábra**). Ehhez kapcsolódóan a szobák közti váltás, vagyis az előző szoba deaktiválása és az új szoba aktiválása is ebben a *Door* komponensben hívódik meg. A *SwitchRoom()* metódus hajtja végre ezt a mechanikát, mégpedig a *Room* script *ActivateRoom()* függvényének a segítségével (**17. kódrészlet**). Ez az aktiválás-deaktiválás egy lista segítségével történik. A *Room* komponensnek van egy enemy listája, amibe bele van húzva minden objektum, ami a szobában található és azt szeretnénk, hogy mindig újra aktiválódjon. Ezután az *ActivaetRoom()* metódusban egy *for* ciklussal végig megyünk a lista elemein és újra aktiváljuk és reseteljük őket. A szobák kialakítása tehát hasonló a tervekhez. A *Room* script tartalmaz egy listát, amelybe *Unity*-ból bele van húzva minden az adott szobában lévő ellenség és akadály.

```
public void MoveToNewRoom(Transform newRoomPoint)
{
```

```
        if (newRoomPoint != null)
            targetPos = new Vector3(newRoomPoint.position.x, newRoomPoint.position.y, t
                                ransform.position.z);
}
```

16. kódrészlet: MoveToNewRoom metódus



33. ábra: Kamera pozíció

```
private void SwitchRoom(Transform roomToActivate, Transform roomToDeactivate)
{
    Room targetRoom = roomToActivate.GetComponent<Room>();
    Room oldRoom = roomToDeactivate.GetComponent<Room>();

    //Kamera mozgás a room kamera pontjára
    if (cam != null && targetRoom.CameraPoint != null)
        cam.MoveToNewRoom(targetRoom.CameraPoint);
    else if (cam != null)
        cam.MoveToNewRoom(roomToActivate); // fallback

    //Room aktiválás/deaktiválás
    targetRoom.ActivateRoom(true);
    oldRoom.ActivateRoom(false);
}
```

17. kódrészlet: SwitchRoom metódus

4.1.7. Logikák összefoglalása

A játék tervezése során az alapvető ötletek, ihletek gyűjtéséhez és kidolgozásához felhasználtam egy videósorozatot[2]. Ebből alapul vettem a scriptjeimhez néhány megoldást és a saját elgondolásomnak megfelelően fejlesztettem tovább a játékomhoz illeszkedően. A játékban felhasznált *Sprite package*-ek[3] egy részét is ez alapján választottam ki, ezért fellelhetőek hasonlóságok a pályaelemek között. A pályák működése, kialakítása és felépítése viszont mind saját kísérletezés és kigondolás eredménye.

4.2. Boss felépítése, működése

A boss ábrázolására egy a játék történetéhez hű, varázsló *Sprite Package*-et választottam (**34. ábra**). Megjelenésben szerintem megfelel a főgonosz borús hangulatának. Ez a csomag lehetővé tette számomra, különböző animációkat készíthessek a boss-hoz, amelyekkel a játékélmény növelése volt a célom. Létrehoztam különböző támadási, futási, ugrási, halál és sebzési animációkat, valamint egy alap *Idle* állapotot is.

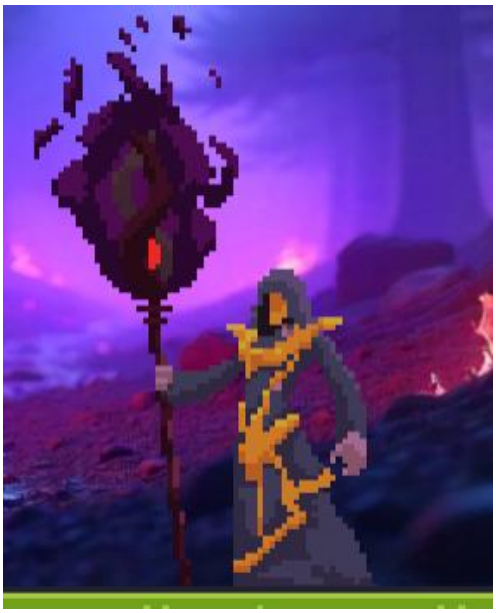
A boss rendszeréhez külön osztályokként, komponensekként alakítottam ki a különböző logikakért felelős részeket ezzel elérve a modularitást és az áttekinthetőséget. Így minden osztály egy jól körülhatárolt funkciót valósít meg. A teljesség igénye nélkül, külön osztályokként kezelem a támadási logikát, sőt ezen belül még az egyes támadások is külön komponensek, továbbá például a fuzzy döntések logikáját, a memóriai tanulást vagy mondhatjuk akár a teleportációt és a mozgást is. Ha minden egy osztályban lenne akkor a kód gyorsan átláthatatlanná válna, főleg egy ilyen komplex rendszer kialakításánál. Ezzel a megoldással a kódok újrafelhasználhatóságát is könnyebbé tettem az esetleges jövőbeli továbbfejlesztésekhez. Továbbá a fejlesztés során így a hibakeresés is egyszerűbb volt, valamint érdemes még az előnyök közé sorolni a rugalmas bővíthetőséget és az egyszerűbb karbantarthatóságot is.

Itt is a teljesség igénye nélkül, de a legfőbb osztályokat és a hozzájuk tartozó néhány, talán legfontosabb változót és metódust megjelenítve készítettem egy diagrammot az egyszerűbb megértés érdekében (**35. ábra**).

A legjelentősebb dolgok, amelyek láthatóak a diagramon, például, hogy a *BossAttackManager* kezeli a támadásokat és a memória komponenst használja a döntéshozatalhoz. A memória komponensem, vagyis a *BossLearningMemory* felelős az olyan adatok tárolásáért, amelyek

[2] https://www.youtube.com/watch?v=TcranVQUQ5U&list=PLgOEwFbvGm5o8hayFB6skAfa8Z-mw4dPV
[3] *Evil Wizard 2, Pixel Adventure 1, Knight Sprite Sheet, Dragon Warrior*

alapján születnek ezek a fuzzy döntések. A *BossAttackBase* egy absztrakt osztály, amelyből leszármaznak a különböző támadások, illetve pluszban van egy Boss*ForceAttack* osztály, ami nem a *BossAttackBase*-ből származik, mert ez más logika alapján működik. A *Health* osztály több komponensen keresztül befolyásolja a boss állapotát az aktuális életerőnek megfelelően. A *BossRoomController* kezeli a boss szobájának adaptivitását a memória és a fuzzy komponensek segítségével. A *BossTeleport* osztály foglalkozik a pozíció váltásokkal fuzzy döntés alapján, a *BossMovement* pedig a főellenség mozgásáért felelős.
A boss logikájának a megértése érdekében létrehoztam egy folyamatábrát is, ami a mellékletek között tekinthető meg (**36. ábra**).



34. ábra: Boss megjelenése

Komponensek közötti kapcsolatok:

- **Asszociáció**:
  - *BossAttackManager – BossLearningMemory*: a menedzser objektum kezeli a boss tanulási memóriáját.
  - *BossMovement – BossAttackManager*: a mozgási komponens ébreszti a támadás menedzsert.
  - *BossLearningMemory – BossAttackBase*: a tanulási objektum kezeli az absztrakt attack osztályt.
  - *BossMovement – Health*: mozgásért felelős osztály kapcsolatban van az életet biztosító objektummal.
  - *BossRoomController – Health*: a boss szobáját vezérlő komponens kezeli az életerővel kapcsolatos osztályt.

- *BossRoomController – BossLearningMemory*: a boss szobáját irányító objektum kapcsolódik a tanuló objektumhoz.
- *BossMovement – BossTeleport*: a mozgási és a teleportálási osztályok kapcsolódnak egymáshoz.
- *BossTeleport – BossLearningMemory*: a teleport objektum kezeli a memória objektumot.
- *BossMovement – BossRoomController*: a boss mozgásának osztálya kezeli a boss szobáját irányító objektumot.

❖ **Öröklődés**:

- *BossAttackDown – BossAttackBase*: a lefele támadás az absztrakt támadási osztályból származik.
- *BossAttackUp– BossAttackBase*: a felfele támadás az absztrakt támadási osztályból származik.
- *BossFireballAttack – BossAttackBase*: az alap támadás az absztrakt támadási osztályból származik.
- *BossDashAttack – BossAttackBase*: a dash támadás az absztrakt támadási osztályból származik.

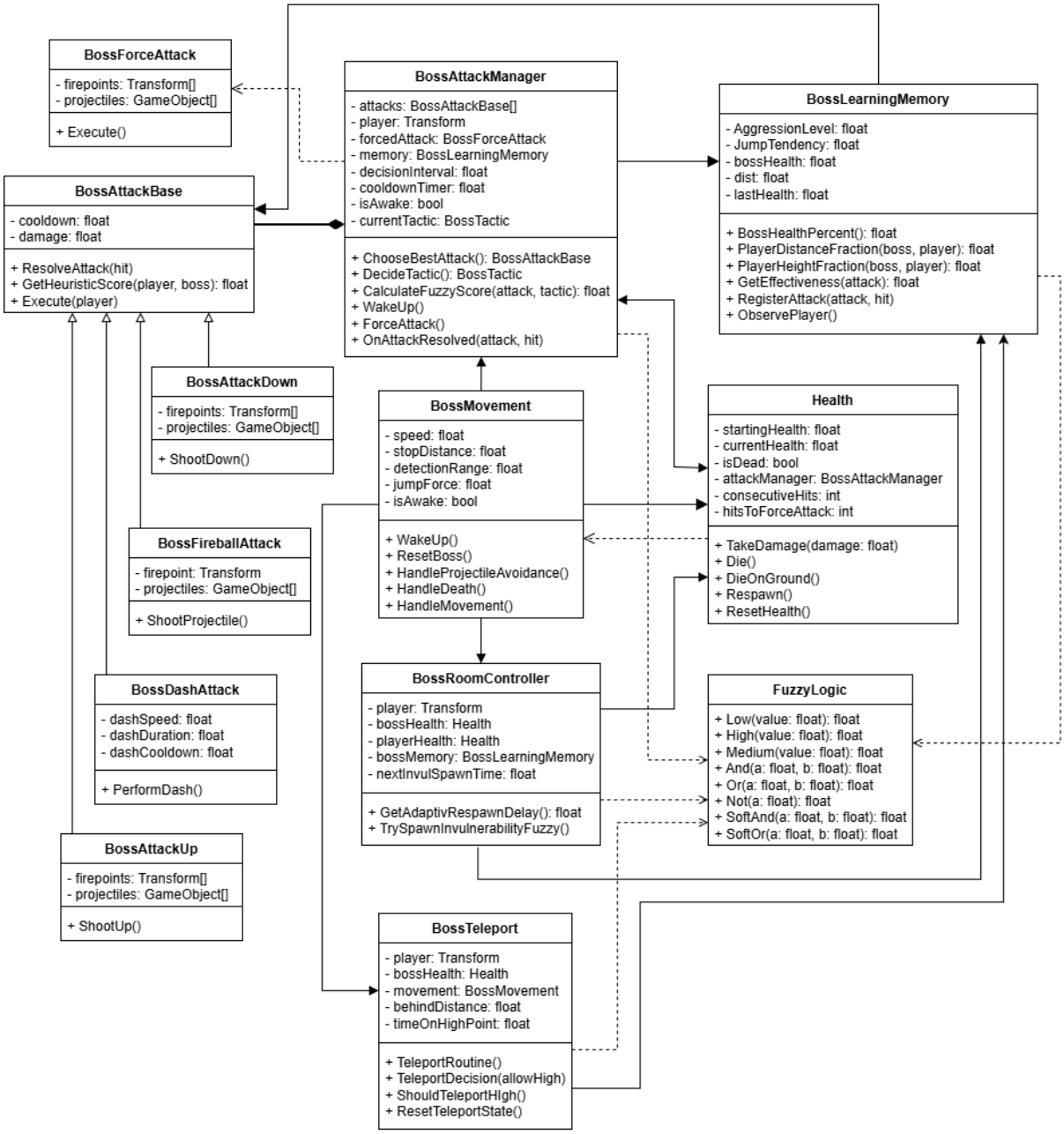
❖ **Dependency (használati kapcsolat)**

- *BossAttackManager – BossForceAttack*: a támadás menedzser használja a boss kényszerített támadás objektumát.
- *BossAttackManager – FuzzyLogic*: a menedzser objektum felhasználja fuzzy logikákat tartalmazó osztályt.
- *BossLearningMemory – FuzzyLogic*: a memória objektum felhasználja fuzzy logikákat tartalmazó osztályt.
- *Health – BossMovement*: az életért felelős osztály felhasználja a boss mozgási komponensét.
- *BossTeleport – FuzzyLogic*: a teleport objektum felhasználja fuzzy logikákat tartalmazó osztályt.
- *BossRoomController – FuzzyLogic*: a boss szobáját irányító objektum felhasználja fuzzy logikákat tartalmazó osztályt.

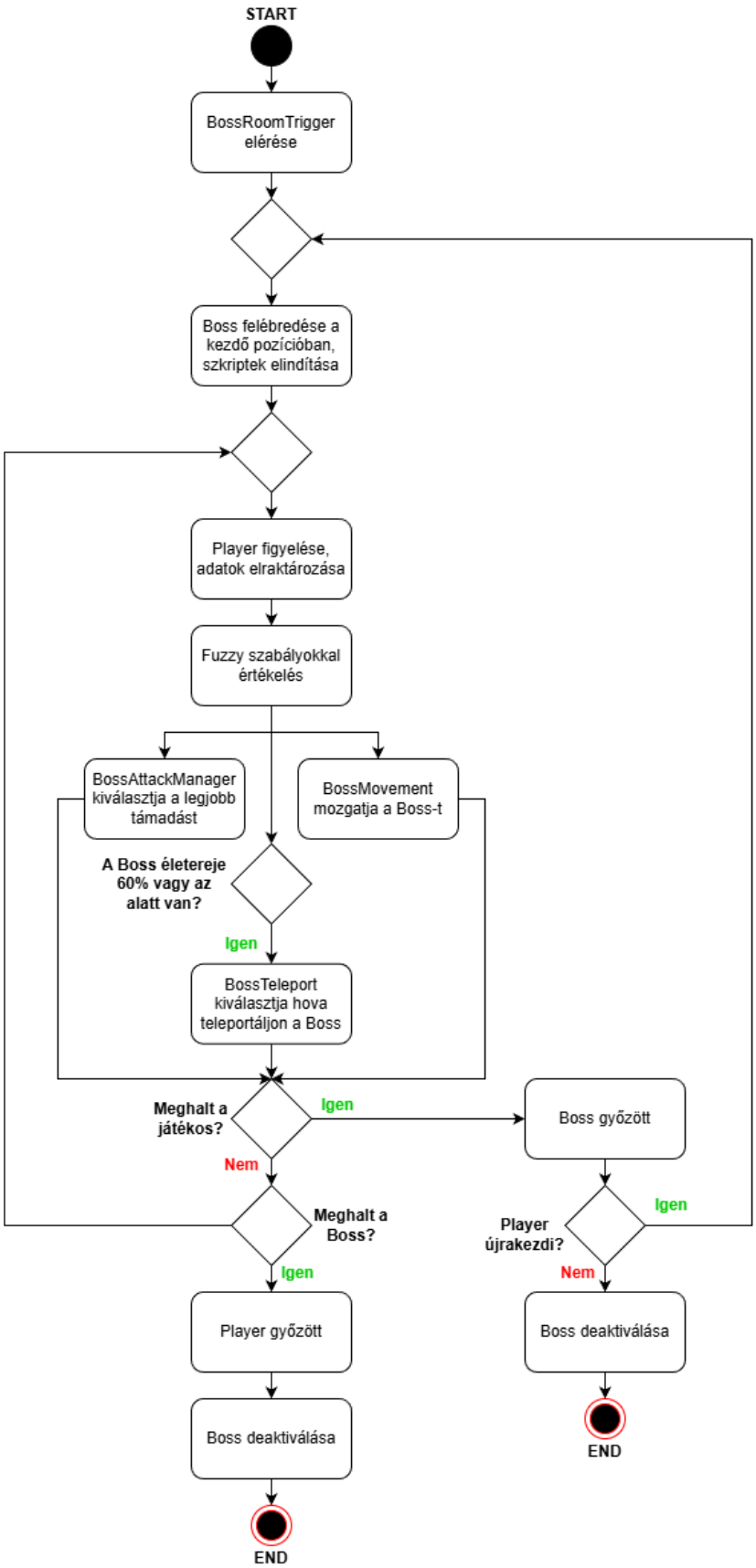
❖ **Kompozíció**

- *BossAttackManager – BossAttackBase*: a támadási menedzser objektum erős tulajdonlási viszonyban van az absztrakt támadási osztállyal.

- **Kétirányú asszociáció**
  - *BossAttackManager – Health*: mindkét osztály kölcsönösen hivatkozik a másikra.



35. ábra: Boss Osztálydiagram



36. ábra: A Boss működésének folyamatábrája

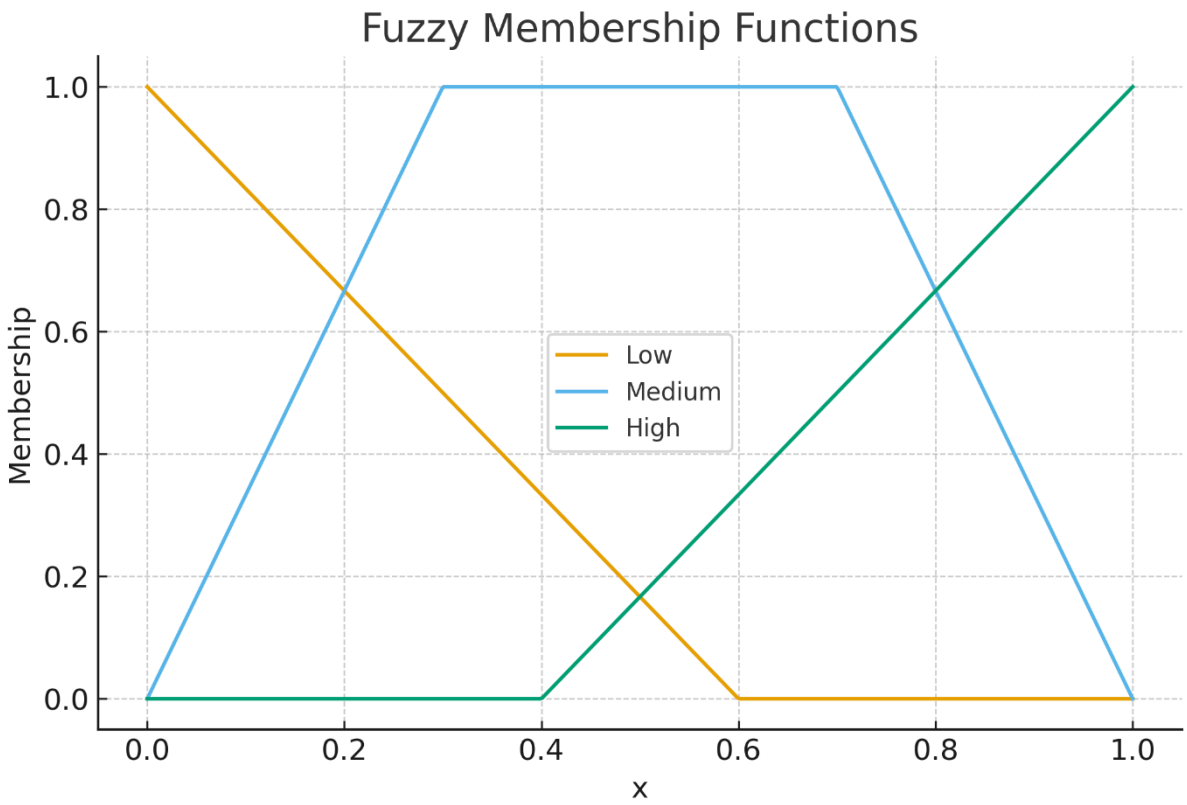
4.3. A végleges Boss AI logika működése

Amint arra korábban is rámutattam, a végleges mesterséges intelligencia szerű logikám tehát egy több komponensből felépülő, fuzzy alapú döntési rendszer lett. Minden komponens önállóan végzi a saját részfeladatát, de egymással szoros adatkapcsolatban működnek. A fő komponensek: *FuzzyLogic*, *BossAttackManager*, *BossLearningMemory*, *BossTeleport*, és *BossRoomController*. Ezek együtt alkotják a főellenfél döntési és tanulási architektúráját. A következő alfejezetekben fel szeretném sorolni, hogy melyik komponensnek mi a szerepe.

4.3.1. FuzzyLogic: az alap motor

A rendszer központi eleme a *FuzzyLogic* osztály, amely az összes fuzzy műveletet kezeli. Itt definiáltam a három fő membership függvényt: Low(), Medium() és High(). Ezek a függvények egy 0 és 1 közötti értéket adnak vissza, attól függően, hogy egy adott bemenet milyen mértékben illeszkedik a logikai kategóriába.

A vizuális szemléltetés érdekében elkészítettem egy Fuzzy Membership függvény ábrát (**37. ábra**), valamint az ábrához tartozó értéktáblázatot (**3. táblázat**).



37. ábra: Fuzzy Tagsági függvények

x (bemenet)	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
Low (narancs)	1.0	0.83	0.67	0.50	0.33	0.17	0.0	0.0	0.0	0.0	0.0
Medium (kék)	0.0	0.33	0.67	1.0	1.0	1.0	1.0	1.0	0.67	0.33	0.0
High (zöld)	0.0	0.0	0.0	0.0	0.0	0.17	0.33	0.50	0.67	0.83	1.0

3. táblázat: Tagsági függvények értéktáblázata

Például a Low(hp) függvény fokozatosan nő, ahogy az életerő csökken, így nem csak azt kapjuk vissza, hogy a HP 'kevés' vagy 'sok', hanem egy köztes fuzzy értéket ad, például 0.73, 0.42 és így tovább.

A fuzzy logika alapműveleteit is itt implementáltam: And(), Or(), Not(). A hagyományos logikai műveletekkel szemben ezek nem bináris módon működnek, hanem súlyozott átlagolással. A SoftAnd() és SoftOr() függvények például nem vágják le az értékeket, hanem finom átmenetet biztosítanak két logikai állapot között.

4.3.2. BossAttackManager: a döntéshozó komponens

A fő döntéshozás a *BossAttackManager* komponensben történik. Ez a script folyamatosan újraértékeli a helyzetet, és fuzzy logika alapján meghatározza a boss aktuális taktikáját. A fuzzy logika nem csupán a bemeneti értékek (HP, távolság, agresszió, ugrási hajlam) kategorizálására szolgál, hanem ezekből épül fel a boss úgynevezett *policy-választó* rendszere is. A „policy" ebben az esetben a Boss AI magasabb szintű viselkedési stratégiáját jelenti (Aggressive, Defensive, Ranged, Evade). A boss tehát nem közvetlenül támadást választ, hanem először fuzzy szabályok alapján taktikát, és csak ezután támadást. A két szint egymástól elkülönül, de szorosan együttműködik.

A rendszer négy fuzzy bemeneti változót használ:

- *hp*: a boss aktuális életereje arányosan (0–1 között),
- *dist*: a játékos és a boss közti távolság,
- *aggr*: az agressziós szint, amelyet a memória számít,
- *jump*: a játékos ugrási hajlama.

Ezen változóknak megfelelően, ahogy már említettem is, fuzzy szabályok alapján dönt a rendszer a taktika típusáról (**18. kódrészlet**).

A mellékletben található példában négy különböző taktikai lehetőség kap fuzzy értéket. A

döntésnél mindig a legmagasabb értékű szabály nyer, ami például azt jelentheti, hogy alacsony HP és közeli játékos esetén a boss „elkerülő” taktikát választ, míg nagy távolságnál vagy nagy ugrási hajlamnál inkább „ranged” módba vált. Ugyanakkor a boss és a player közötti kis távolság és az agressziós szint alacsony állapota esetén a „védekező” taktika kerül kiválasztásra, míg magas agresszió és magasabb HP esetén az „agresszív” taktika lesz a használt mód.
A kiválasztott taktika után a *CalculateFuzzyScore()* függvényem minden támadási típushoz kiszámolja a fuzzy pontszámot (**19. kódrészlet**). Ez a pontszám a távolság, az agresszió, a sikerességi arány és egy kis véletlen szorzó kombinációjából áll össze. Az így kiszámolt érték alapján a boss nem mindig ugyanazt a támadást választja, hanem a legvalószínűbb, de nem garantáltan legjobb opciót hajtja végre.

4.3.3. BossLearningMemory: adaptív tanulás és statisztika

Ahogy ezt is kifejtettem már korábban, a tanulási folyamatot a *BossLearningMemory* komponens végzi. Úgy gondolom ez itt is releváns, ezért kerül újra megemlítésre. Ez nem közvetlenül döntéseket hoz, hanem adatokat gyűjt, amelyeket a többi komponens használ fuzzy bemenetként. A memória figyeli a játékos mozgási mintáit, például, hogy mennyit ugrik, mennyire agresszív, és milyen gyakran okoz sebzést. Minden támadás után rögzíti, hogy az sikeres volt-e vagy sem. A komponens egy egyszerű statisztikai tanulási modellt használ, ha egy támadás hatékony, a súlya nő, ha hatástalan, fokozatosan csökken.
Ezen kívül minden frissítéskor kiszámolja az *Update*() metódus a *jumpTendency* (ugrási hajlam) és *aggressionLevel* értékeket, az *ObservePlayer*() metódus meghívásával (**20. kódrészlet**). Az *ObservePlayer*() metódus logikájának a megértése érdekében is beszúrtam egy rövid kódot (**21. kódrészlet**). A fuzzy logika ezekből dolgozik, például a *jumpHigh* változó a memória alapján dől el, nem véletlenből. A memória lassan felejt, a változók *decay rate*-tel csökkennek, így a boss fokozatosan alkalmazkodik a játékos új stílusához (**22. kódrészlet**).

4.3.4. BossTeleport: pozícióváltoztatás fuzzy alapon

A *BossTeleport* komponens dönti el, mikor és hová teleportál a főellenfél. Ezt a döntést is fuzzy szabályok határozzák meg. A *BossTeleport* komponens *ShouldTeleportHigh()* függvénye gondoskodik arról, hogy eldöntse mikor kellene a boss-nak magasabb pozícióba menekülnie (**23. kódrészlet**). Ennek a lényege valójában az, hogy amikor a boss „nagyobb bajba” kerül, akkor egy olyan helyre teleportáljon, ahol inkább biztonságban lehet. Ennek a döntésnek a súlyozásával foglalkozik ez a metódus.
A logika lényege tehát, hogy ha a boss alacsony életerőn van, sok sebzést kapott, vagy a játékos

agresszívan, ugrálva támad, akkor nő az esélye, hogy a boss magas pontra teleportál. Ez a mozdulat védekező taktika, ami időt ad neki a regenerálódásra vagy újratervezésre. Ha viszont az agresszió magas és a boss élete nem vészesen alacsony, akkor inkább a játékos mögé teleportál, hogy közelharcot kezdeményezzen.

4.3.5. BossRoomController: adaptív harci környezet

A *BossRoomController* a harci aréna állapotát irányítja fuzzy szabályok alapján. A rendszer itt is 0 és 1 közötti fuzzy értékeket használ, hogy finoman szabályozza a pickup itemek (élet és invulnerabilitás) megjelenését.

A health pickup újra spawnolásánál például a *GetAdaptiveRespawnDelay()* metódus a fuzzy szabályok alapján kiszámolja a következő spawnolási időt és ennek megfelelően állítja be *HealthPickupRespawnRoutine()* metódusom a következő *respawnDelay* -t (**24. kódrészlet**).

Ha a boss gyenge, a játékos nagyon aktív, sokat ugrál, vagy több halál is történt már a boss fight alatt, akkor a fuzzy érték nő, és rövidebb idő után újra megjelenhetnek a gyógyító elemek. Hasonló elven működik az invulnerability pickup is, csak itt több paramétert figyelek, ezek a játékos életereje, a boss agressziója, a halálozások száma és a korábbi körök időtartama. Ezekkel a fuzzy szabályokkal dinamikusan változik a harc nehézsége, ha a játékos túl könnyen győz, a rendszer fokozatosan kevesebb segítséget ad, ha viszont nehezen boldogul, enyhén kedvezőbb feltételeket teremt a sikeresebb végrehajtás érdekében.

4.3.6. Az összetett logika működése

Egyben nézve tehát a boss Fuzzy alapú mesterséges intelligencia szerű logikája a következő:

A harc minden pillanatában több komponens egyszerre dolgozik. A folyamat a játékos viselkedésének megfigyelésével indul. A *BossLearningMemory* rögzíti az ugrásokat, sebzéseket és támadások sikerességét, majd frissíti a statisztikai változókat. Ezek az értékek közvetlenül a fuzzy bemenetekbe kerülnek.

A *BossAttackManager* folyamatosan új értékelést végez. A fuzzy logika alapján meghatározza, melyik taktika érvényesüljön, például agresszív, védekező vagy távolsági. Ekkor minden támadástípust külön pontoz, és kiválasztja azt, amelyik a legmagasabb fuzzy *score*-t kapta. Az így kiválasztott támadást a boss azonnal végre is hajtja.

Ha a játékos taktikát vált, például kevesebbet ugrál vagy távolságot tart, a memória gyorsan leköveti ezt. A fuzzy inputok fokozatosan változnak, ami miatt a boss taktikája sem vált hirtelen, hanem organikusan igazodik. A fuzzy logika folyamatosan interpolál a régi és az új értékek között, így a viselkedés emberibbnek, természetesebbnek hat.

Közben a *BossTeleport* és a *BossRoomController* is párhuzamosan fut. A teleport rendszer a boss állapotát figyeli, és fuzzy logika alapján dönt, mikor és hova pozícionálja újra magát. A környezetkezelő rendszer adaptívan szabályozza az aréna nehézségét. A komponensek egymás adatait használják, de nem közvetlenül hívogatják egymást, hanem megosztott értékekből dolgoznak. Ezzel a rendszer moduláris és jól karbantartható maradt.

4.4. Tesztek

Ebben az alfejezetben a fejlesztés végén a teszteléshez használt logikákat szeretném bemutatni. *Unity*-ban kétféle tesztet lehet végezni, az egyik a *Play Mode* tesztek, amelyekkel a játék futása közbeni funkciókat lehet tesztelni, a másik pedig az *Edit Mode* tesztek, amelyekkel olyan logikákat lehet tesztelni, amelyek nem igénylik, hogy fusson a játék. A legalapvetőbb logikák működésének az ellenőrzésére készítettem teszteket.

4.4.1. Play Mode tesztek

Először is a *PlayerMovement* komponensre írtam teszteket, hiszen a játékos alapvető irányítási mechanikáit szerettem volna ellenőrizni. Készítettem tesztet a szimpla és a dupla ugrás logikájára, a jobbra és balra mozgásokra, valamint a guggolásra is. Ezen mechanikák mindegyike sikeres működést mutatott a tesztek alapján is (**38. ábra**). Ahhoz, hogy ezeket a teszteket el tudjam végezni, a *PlayerMovement* komponenshez hozzá kellett adnom néhány segítő metódust, hiszen a script védelmi szintjei miatt, másképpen nem tudtam volna futtatni a teszt scriptet.

Példának okáért megmutatnék egy egyszerű Play Mode tesztet, amely a player balra mozgásának a működését ellenőrzi (**25. kódrészlet**).

Segédmetódusként a *TestHorizontalInput()*-ot belettem a *PlayerMovement* scriptbe és amikor ez a *useTestingInput* értékét igazra állítja, akkor a mozgáshoz az *Input.GetAxisRaw()* helyett a tesztben megadott statikus értéket használja.

A *PlayerDeathTests* tesztosztályom a játékos halál logikájának különböző funkcióit ellenőrzi (**39. ábra**). A *PlayerDies_WhenHealthReachesZero* metódus azt vizsgálja meg, hogy amikor a játékos életereje nullára csökken, akkor valóban meghal-e. Ehhez a *Health* komponens egy metódusát hívja meg a következőképpen: *TakeDamage(health.currentHealth),* ezzel biztosítja, hogy az összes aktuális élet levonódik. Ezután ellenőrzi, hogy az *IsDead()* függvény visszaadja-e az igaz értéket, valamint az aktuális életerőt is ellenőrzi, hogy nulla lett-e. Egy másik tesztel azt vizsgáltam, hogy a player halála esetén minden komponens leáll-e, amely bele van húzva a komponensek listájába. Ez a teszt *PlayerComponentsDisabledOnDeath* metódussal valósul

meg. Ez igazából nem kifejezetten a player-re vonatkozó teszt, hiszen ez az összes életerővel rendelkező objektumnál ugyanígy működne, vagyis, ha itt működik, minden további objektumon is működnie kell. A teszt során minden komponens *enabled* állapota ellenőrzésre került, hogy hamis-e. A helyes mechanika szerint a player halála után meg kell jelennie a *GameOver Screen*-nek. Ennek a megvalósulását hivatott letesztelni a *GameOverScreenActivatesOnPlayerDeath* metódus. A teszt az elvárt működéshez hűen vár egy másodpercet a player halála után majd ellenőrzi, hogy aktiválódott-e a *GameOver* képernyő.

A főellenségem felébresztése és az egész boss szoba aktiválása a játékomban egy pont elérésekor történik, ez a *BossRoomTrigger.* Az ezen lévő script ébreszti fel a boss-t és újra tölti a player életeit, hogy a kezdeti életerővel kezdhesse el a harcot. Ezen logikák tesztelésére szolgál a *BossRoomTriggerTests* komponens *Trigger_ActivatesBossAndRestoresPlayerHealth* metódusa (**26. kódrészlet**). A *Setup()* függvényben betöltjük a Level1 Scene-t, megkeressük a player-t és annak *Health* komponensét, megkeressük és aktiváljuk a boss-t, valamint a biztonság kedvéért visszaállítjuk a *BossRoomTrigger*-t is. A teszt metódusban lecsökkentjük a player életerejét, hogy megtudjuk vizsgálni visszatöltődik-e amikor belemegy a trigger-be. Továbbá ellenőrizzük, hogy a boss valóban felébredt, vagyis aktiválódott-e.

4.4.2. Edit Mode tesztek

Az életerőrendszer működésének ellenőrzésére készítettem néhány teszt metódust, amelyek *Edit Mode*-ban működnek (**40. ábra**). Ezek minden animáció és UI nélkül csak kimondottan a logikát tesztelik. Van egy teszt, ami arra szolgál, hogy megmutassa túlléphető-e a maximum életerő. Ez nem más, mint az *AddHealth_DoesNotExceedStartingHealth* metódus, aminak az kell legyen az eredménye, hogy nem léphetjük túl. A *ResetHealth_RestoresFullHealth* függvény azt ellenőrzi, hogy a *ResetHealth()* visszatölti-e a kezdeti életerőt. A következő metódus a *TakeDamage_DoesNotGoBelowZero*, amely arra szolgál, hogy teszteljük az életerő negatívba való áttérésének lehetetlenségét. Továbbá teszteltem az *AddHealth()* metódus helyes működését, valamint a *ResetPlayerState()* mechanikáját is.

Kicsit az ellenségek logikáját is szerettem volna tesztelni *Edit Mode* tesztekkel. A *RangedEnemy* komponens *FindInactiveProjectile()* metódusa végig megy az EnemyProjectile tömbön és megnézi, hogy van-e inaktív, ha talál, akkor visszaadja, de ha nem talál, akkor a nulladik indexűvel tér vissza, ezzel biztosítva, hogy mindig talál lövedéket. Mindkét esetet leteszteltem egy egyszerűsített esetben (**27. kódrészlet**). Mindkét teszt sikeres volt.

Ezt a logikát nem csak a *RangedEnemy* scriptemben használtam, hanem mind a player, mind a boss esetében is. Mivel itt a tesztek alapján is kimondható, hogy jól működik, ezért kijelenthető,

hogy a másik objektumoknál is helyes a működés.

4.5. Fejlesztéshez használt eszközök

A játék fejlesztése során használt eszközöket a következő táblázatban gyűjtöttem össze (**4. táblázat**).

Használt eszköz	Típus	Leírás
Unity 2022.3 LTS	Játékmotor	Mérnökinformatikus tanulmányaim során már használnom kellett ezt az eszközt és mivel élveztem vele a munkát, valamint kellően sokszínű funkciókkal ellátott rendszernek találom, ezért ezt választottam a szakdolgozatomhoz is.
Unity Asset Store	Asset store	Innen szereztem be a legtöbb alapanyagot, például a pálya szinte minden elemét, valamint a player, az enemy és a boss karaktereket is.
Unity komponensek	Rigidbody, Collider, Animator	A fejlesztés során gyakran használtam a Unity ezen beépített eszközeit.
C#	Programnyelv	Ez a nyelv szorosan integrálódik a Unity környezetébe, így a komponensek közötti kommunikáció, eseménykezelés és állapotmenedzsment könnyen megvalósítható volt.
Visual Studio Code	Kódszerkesztő alkalmazás	A kódot elsősorban Visual Studio Code-ban írtam, amelyet a Unity automatikusan összekapcsol a projekttel. A fejlesztés során külön script fájlokat hoztam létre az egyes játékelemekhez, például a játékos irányításához, az ellenfelek viselkedéséhez és a különféle csapdák működéséhez is.
GitHub	Verziókezelő alkalmazás	A verziókezeléshez GitHubot használtam. A projekt elején létrehoztam egy privát tárolót, ahol a teljes forráskódot és az erőforrásokat kezeltem. Valamint a végleges kódokat is egy GitHub repository-ban osztottam meg.
Freepik Pikaso AI Image Generator	AI kép generátor	A különböző szobák háttereihez, valamint a például a főmenü hátteréhez is AI segítségével generált képeket használtam.
Draw.io	Ábra készítő alkalmazás	Az eszköz drag-and-drop felülete egyszerűvé tette az elemek rendezését és összekapcsolását, a diagramokat pedig exportálás után képfájl formátumban közvetlenül a szakdolgozatomba tudtam illeszteni.

4. táblázat: Fejlesztéshez használt eszközök

5. Eredmények

5.1. Saját értékelés

Az eredeti elgondolásomnak megfelelően egy olyan 2 dimenziós platformer játékot szerettem volna létrehozni, amelyben megtalálom az egyensúlyt a játékos sikerélmények a biztosítása és a kellő felbosszantása között. A projekt célja tehát egy rage game típusú játék megvalósítása volt.

A rage game olyan ügyességi játék, amelynek legfőbb jellemzője a túlzott nehézség és a frusztráló játékmenet. A játékos gyakran kis hibák miatt veszít jelentős előrehaladást, ami erős érzelmi reakciót válthat ki. Ezt a feszültséget szándékosan építettem bele a játékba. A célom nem csupán a siker elérésének a biztosítása volt, hanem a játékos türelemének és kitartásának a próbára tétele is. Egy ilyen típusú játék fejlesztése technikailag egyszerűbbnek tűnhet, de a pontos időzítések, visszajelzések és fizikamechanikák kulcsszerepet kapnak. A játékosnak mindig úgy kell éreznie, hogy a hibáért ő felelős, nem pedig a rendszer. Ezt a balanszot akartam elérni a fejlesztés során, vagyis, hogy a nehézség legyen kihívás, ne igazságtalanság.

A nehézség megtervezése során rengeteg kísérletet végeztem amire elértem a kívánt állapotot. A pályák kialakításakor figyeltem arra, hogy minden akadály tanulható legyen, de az első próbálkozásoknál szinte biztosan hibázzon a játékos. A cél az volt, hogy a kudarc után azonnal felismerhető legyen, mit kellett volna másképp csinálni. Meglátásom szerint ez a fajta tanulási ív adja a rage game-ek sajátos „flow”-élményét, amikor a frusztráció fokozatosan átfordul sikerélménybe. Külön figyelmet fordítottam a visszajelzésekre. A karakter mozgása, az ugrás sebessége, a hanghatások mind befolyásolják, mennyire érezhető igazságosnak a játék. Többször módosítottam a fizikai beállításokat és az akadályok elhelyezését, mire elértem azt a pontot, ahol a nehézség éppen elégnek tűnt ahhoz, hogy kihívást jelentsen, de ne váljon lehetetlenné.

A játékomban az előrehaladás szobáról szobára történik. A kamera mozgást is ennek megfelelően alakítottam ki, amikor a játékos egy új szobába lép be, akkor a kamera is átugrik arra szobára és ilyenkor töltődnek be az ott található objektumok. Jelenleg 7 különböző szoba található a játékban. Minden szoba más és más kihívás elé állítja a játékost. Igyekeztem ezeket úgy kialakítani, hogy a játékos ne monotonitást érezzen az előrehaladás során, hanem mindig újabb mechanikákat kelljen elsajátítania ahhoz, hogy teljesíteni tudja kihívásokat. Az első hat szobában különféle akadályok és egyszerűbben legyőzhető ellenségek találhatóak, a hetedik szobában pedig a boss helyezkedik el, ami a játék legnehezebb próbatétele elé állítja a játékost. Ahogy azt már említettem, a játék szerkezete szobákra épül, ezért fontos volt, hogy a

memóriahasználat ne növekedjen feleslegesen. Ennek érdekében mindig csak az aktuális szoba töltődik be, a korábbiak pedig inaktiválódnak, amint a játékos tovább halad. Ez a megoldás segített elkerülni a felesleges objektumterhelést, és érezhetően javította a teljesítményt.
A fejlesztés egyik legidőigényesebb része az animációk finomhangolása volt. A karakter mozgását, ugrását és halálanimációját többször módosítottam, mire összhangba került a játékmenet ritmusával. Apró változtatások is sokat számítottak, például az ugrás előtti késleltetés vagy az animáció sebessége erősen befolyásolta a játékérzetet.

5.2. Játékosok visszajelzései

Miután elkészült a játékom egy már ténylegesen működő, végig játszható verziója, megkértem néhány embert, hogy próbálják ki és mondják el a tapasztalataikat, véleményüket. A reakciók összeségében pozitívok voltak. Szerintük is sikerült elérnem az egyik fő célom, ami egy olyan rage game típusú játék létrehozása volt, amely képes egyensúlyban tartani a sikerélményeket és a játék kiismerési folyamata során kialakuló bosszankodást.
A játékomba beleépítettem egy *deathcounter* paramétert, ami folyamatosan frissül és végig látható egyfajta visszajelzésként a játékosok számára is. Átlagosan az jött ki eredményül, hogy amikor egy ember anélkül, hogy bármit tudna a játékról, leül és megpróbálja végig vinni azt, körülbelül 30 halál és hozzávetőlegesen 20-25 perc játékidő után sikeresen tudja abszolválni a kihívást. Persze ez egy olyan ember esetében releváns, aki azért hozzá van szokva a számítógépes játékokhoz. Kíváncsiságból ugyanis olyan embereket is megkértem, hogy próbálják ki, akik talán még soha nem találkoztak hasonló játékkal. Nekik a tesztelések alapján sokkal inkább nehéznek bizonyult a játék, mint élvezhetőnek. Azonban alapvetően célközönségnek nem is ezeket az embereket szerettem volna megcélozni, hanem a játékos közösséget.

- Alapvetően a játékmechanika lehetővé teszi a falra mászást, illetve a különböző objektumok, platformok, blokkok oldalán történő megkapaszkodást. Többen jelezték, hogy számukra ez szokatlan és inkább negatívumként hat, mint segítségképpen. Ugyanakkor én erre a mechanikára építettem fel több akadályt, illetve elérendő pozíciót is, ezért elfogadtam a kritikájukat, de nem változtattam ennek megfelelően a játékon.
- Továbbá szintén rámutattak arra, hogy amikor meghal a játékos és feljön a *DeathScreen*, a kiválasztható gombok sorrendje kicsit frusztráló, hiszen legfelül van a *Restart* gomb és alatta helyezkedik el a *Load checkpoin*t lehetőség (**41. ábra**). Ezzel az volt szerintük a probléma, hogy amikor a játékos meghal és feljön ez a screen, akkor automatikusan

először a legfelső opción fog állni a választó objektum és ha valaki nem elég körültekintő, akkor hajlamos egyből kiválasztani azt, és így az addig megszerzett checkpointok elvesznek és újra kell kezdeni a játékot. Ez is egy jogos meglátás volt, viszont miután mérlegeltem a problémát, úgy döntöttem, hogy mivel ez egy rage game típusú játék, ezért ezt sem változtatom meg teljesen, annak érdekében, hogy ezt is meg kelljen tanulnia a játékosnak a saját hibáján, ha arra kerül sor. Segítségképpen viszont annyit módosítottam ezen, hogy mindig ott fog állni a választó nyíl, amit legutóbb kiválasztottak. Ennek megfelelően, ha a játékos egyszer már kiválasztotta a checkpoint opciót, akkor legközelebb már sokkal kisebb eséllyel nyom rá véletlenül az újrakezdés gombra.

- Kiemeltek néhány pozitívumot is. Többen jelezték, hogy a felvehető objektumok, mint például a plusz élet és a pajzs, sokat segítenek, és szerintük jól vannak elhelyezve a játék során. Nagyobb sikert aratott még a leeső platform csapdám, amely, amikor először találkoztak vele a játékos kevésbé volt örömteli élmény, de szórakoztatónak és jó ötletnek tatálták.
- A boss fight is elérte a kívánt hatást. Elmondható, hogy ez szinte mindenkinek, aki kipróbálta és eljutott idáig nagy kihívást jelentett, de a sikertelen próbálkozások során egyre inkább azt az érzést tapasztaltam rajtuk, hogy minél inkább csak meg akarták oldani ezt a feladatot is és nem feladták a frusztráció és a stressz miatt. Végül több ember is sikeresen tudta venni ezt az akadályt is, így elmondható, hogy a játék teljesíthető.

5.3. Eltérés a tervtől

Néhány helyen különböző okok miatt eltértem az eredeti tervektől. Kezdetben például azt terveztem, hogy az AI logikát akár több ellenség vezérlésére is használnám, nem csak a boss esetében. Ezen elképzelésem szerint egy egységes, moduláris rendszer vezérelhette volna az ellenfeleket, beleértve a boss-t is. A játék kialakítása során azonban arra az álláspontra jutottam, hogy ez lehet már túl bonyolítaná és inkább kissé negatív irányba vinné el azt a platformer élményt, amelyet én a játékommal nyújtani szeretnék. Ezért arra a döntésre jutottam, hogy a játék jelenlegi struktúrájában nem használom ezt más ellenségeknél. A célom az lett, hogy először sikeresen bele építsem a boss logikájába ezt a mechanikát és ha beválik, akkor majd a későbbiekben fel lehet használni ezt más ellenségeknél is.

Ennek megfelelően el is készítettem egy Fuzzy logikával működő boss rendszert, ami a jövőben egyszerűbben be építhető bármely más ellenfél mechanikájába. A boss elleni harc ideális terepet

teremtett ahhoz, hogy ezt a logikát kikísérletezhessem és tesztelhessem, hiszen itt a folyamatosan változó számos körülménynek megfelelően kell gyorsan döntéseket hozni. A fuzzy logika alkalmazásával lehetővé vált, hogy ne bináris módon (például „közel” vagy „távol”) legyenek kezelve az értékek, hanem egy folyamatos skálán, amivel elérte, hogy a boss viselkedése természetesebb és kiszámíthatatlanabb lett.
Fontos azonban megjegyezni, hogy a rendszer kialakítása során jelentős figyelmet fordítottam az újrafelhasználhatóságra és a komponens alapú felépítésre. Ahogyan azt már korábban tárgyaltam, az egyes funkciókat különböző osztályokba szerveztem ki, mint például a *BossAttackManager*, *BossLearningMemory* vagy a *FuzzyLogic.* Ezzel a megoldással a logika nem kötődik kizárólagosan a boss-hoz, hanem a fuzzy döntéshozatal, a tanulási memória a támadási lehetőségek pontozása olyan modulok lettek, amelyek kisebb módosítások után beépíthetőek további ellenségek viselkedésébe is.

Példának okáért, egy másik ellenség típus esetében mondjuk ez így nézhetne ki:

- Létre kellene hozni egy új *AttackManager* komponenst, amelyet az adott ellenség saját támadásainak megfelelően kellene kialakítani.
- Változtatni kellene a *LearningMemory* osztályon is, hogy az csak a releváns paramétereket tartalmazza, például a távolságot vagy a sebzések hatékonyságát.
- A jelenlegi *FuzzyLogic* kialakításán pedig valószínűsíthetőleg szinte nem is kellene változtatni, hiszen ez teljesen független az aktuális karaktertől.

A boss-hoz kialakított architektúra tehát nem egy zárt rendszer, hanem inkább egyfajta tesztplatformként szolgált a Fuzzy logika alapú AI megoldásokhoz és ez később egyszerűen kibővíthető és általánosítható. Az így létrehozott komponens alapú megközelítésnek köszönhetően ez a döntési logika kisebb ellenfeleknél is alkalmazható mondjuk támadási távolság, sebesség vagy például menekülési viselkedés meghatározásához.
Összeségében tehát, annak ellenére, hogy a jelenlegi implementáció kifejezetten a boss elleni harchoz készült, olyan megoldások állnak mögötte, amelyek a későbbiekben könnyen átépíthetőek további ellenség típusoknak megfelelően. Ezzel a megközelítéssel biztosítottam a fejlesztés jövőbeli bővíthetőségét és a rendszer hosszabb távú fenntarthatóságát is.

6. Konklúzió, jövőbelli tervek

6.1. Összegzés, értékelés, tapasztalatok

Úgy gondolom a játék fejlesztése során sikerült egy jól strukturált, modulárisan felépített játékrendszert létrehoznom. Véleményem szerint ezzel sikerült elérnem, hogy a komponensek könnyedén átláthatóak, karbantarthatóak és meg van bennük a jövőbeli tovább fejleszthetőség lehetősége.

Elmondható, hogy a fuzzy logika használatával a boss viselkedése oly módon valósult meg, hogy az sokkal természetesebb és kifinomultabb lett, mint lehetett volna egy hagyományos megoldással. Egy olyan rendszert sikerült létrehozni, amely képes valós időben reagálni a játékos viselkedésére és ezalapján, valamint az előző támadások hatékonysága alapján hozza meg a döntéseit és változtatja a taktikáját. Ezáltal a harc adaptív és dinamikusabb lesz, amely intenzívebb játékélményt nyújt.

Egy teljes értékű játékprogramot szerettem volna létrehozni, amellyel izgalmas és kellően kihívó játékélményt tudok nyújtani a játékosok számára és úgy gondolom ezt sikerült megvalósítanom. Sok funkciót már a tervezés során kitaláltam, hogy bele szeretnék építeni majd a játékba, de akadtak olyan ötletek is, amelyek csak a fejlesztés során jutottak eszembe és azoknak megfelelően módosítva a kezdeti terveket alakítottam ki a jelenlegi verziójú játékot. Kijelenthetem, hogy egy ilyen játék megvalósítása bonyolultabbnak és összetettebbnek bizonyult, mint ahogy én azt először gondoltam. Hatékony megoldásokkal és néhol kisebb kompromisszumok megkötésével sikeresen létrejöhetett a játékom. A játék tervezése és fejlesztése során olyan tapasztalatokat tudtam szerezni és olyan képességeket tudtam elsajátítani, amelyek még a jövőben biztos hasznomra lesznek.

6.2. Jövőbeli tervek

A játék működéséhez a legalapvetőbb funkciókat már ez a verzió is tartalmazza, azonban ez közel sem jelenti azt, hogy a játékot ne lehetne további fejlesztésekkel kiegészíteni a jövőben.

Jelenleg a játékban 7 különálló szoba van, de valójában ez tekinthető egy szintnek, mondjuk az első szintnek. A jövőben beépíthető lenne a boss fight után egy olyan lehetőség, hogy a sikeres legyőzés után nem véget érne a játék, hanem megnyílna egy ajtó, ami átdobja a játékost egy második szintre. A játéktér ott is hasonlóan nézhetne ki, vagyis szintén szobákra épülő megoldást képzelek el. A különböző termek azonban itt már tartalmazhatnának összetettebb, bonyolultabb pályaszerkezeteteket és nehezebben leküzdhető akadályokat és ellenségeket. A második szint végén szintén lehetne egy utolsó szoba, ahol egy még összetettebb logikával

ellátott boss várhatná a játékost. Ez a boss már felhasználhatná esetleg az első szinten legyőzött boss tapasztalatait és ezeket megvizsgálva, például, hogy mi működött a játékos ellen és mi nem, egy még kifinomultabb és jelentősen nagyobb kihívást jelentő boss ellenei harcban vehetne részt a játékos.

Ahogy azt már egy előző fejezetben is fejtegettem, a boss működéséhez felhasznált fuzzy logika alapú megoldás felhasználható lehetne más, egyszerűbb ellenségekhez is. Egy másik továbbfejlesztési lehetőségnek tartom akár egy teljes fuzzy alapú ellenség irányítási rendszer kialakítását is. Ebben a megoldásban egy központi memória komponens tudná kezelni az összes megfigyelt paramétert és minden ellenség ebből kitudná venni a saját döntési mechanikáihoz, fuzzy tagsági függvényeihez szükséges értékeket. Ez a mód még nagyobb adaptivitást vihetne a játékba és nem csak a boss harc során tapasztalhatna a player ennyire intenzív játékélményt. A boss szobájához felhasznált logikát is ki lehetne terjeszteni a többi teremhez. Így azok is adaptívan alkalmazkodhatnának a játékmenethez, akár a felevehető extra képességek tekintetében. Tovább gondolva ezt, nem tartom megvalósíthatatlannak azt sem, hogy ezen fuzzy logika alapján a szobák is adaptívan alakuljanak ki a játék során. Tehát hogy ne a most is használt előre kialakított szobák legyenek benne a játékban, hanem mondjuk az első szint teljesítése alapján elraktározzuk az információkat a különböző pályaelemekről, hogy mennyi próbálkozás után sikerült teljesíteni őket például. Ezután pedig eldönthetnénk, hogy egy nehezebb pályastruktúrát szeretnénk kialakítani és azokat az akadályokat használni fel többször, amelyek nehezebbnek bizonyultak vagy pedig inkább kedveskedni szeretnék a játékosnak és a tapasztalatok alapján egyszerűbb pályákat kialakítani neki.

A jelenlegi jutalmazási rendszert is tovább lehetne fejleszteni a jövőben. Ennek egy megközelítése lehetne például, annak érdekében, hogy több értelmet adjunk a pontszerzésnek, hogy bizonyos képességek vagy itemek akkor jelenjenek meg a játékban, amikor a player elér egy bizonyos küszöbértéket a pontszámával. Ilyen módon akár megvalósíthatónak tartom például, egyúttal a főhős karakter fejlődésének a biztosításával, hogy a támadási formák egyre kifinomultabbak, hatékonyabbak lehetnének ahogy minél több pontot szerez a játékos. Jelenleg a főhősnek egyféle támadási módja van, ez pedig egy fix irányba történő lövés, ami csak a játékos mozgatásával irányítható. A jövőben viszont létre lehetne hozni egy külön logikát a lövedékek irányítására vagy másféle támadások, például közelharc vagy valami egyéb, nehezebb támadási lehetőség kialakítására. Ilyen mechanikák implementálásával tovább színesíthető lenne a játék.

Irodalomjegyzék

[1] Csikszentmihalyi M.: Flow: The Psychology of Optimal Experience, Harper & Row, 1990. ISBN: 9780060162535

[2] Koster R.: A Theory of Fun for Game Design, O'Reilly Media, 2004. ISBN-13: 9781932111972

[3] Salen K., Zimmerman E.: Rules of Play: Game Design Fundamentals, MIT Press, 2004. ISBN: 9780262240451

[4] Swink S.: Game Feel: A Game Designer's Guide to Virtual Sensation, Morgan Kaufmann, 2009. ISBN: 9780123743282

[5] Gee J. P.: What Video Games Have to Teach Us About Learning and Literacy, Palgrave Macmillan, 2007. ISBN: 9781403984531

[6] Donovan T.: Replay: The History of Video Games, Yellow Ant, 2010. ISBN: 9780956507204

[7] Juul J.: Handmade Pixels: Independent Video Games and the Quest for Authenticity, MIT Press, 2019. ISBN: 9780262042796

[8] Kent S. L.: The Ultimate History of Video Games, Prima Publishing, 2001. ISBN: 9780761536437

[9] Kohler C.: Power-Up: How Japanese Video Games Gave the World an Extra Life, Dover Publications, 2016. ISBN: 9780486816425

[10] Ryan J.: Super Mario: How Nintendo Conquered America, Portfolio, 2011. ISBN: 9781591844051

[11] Wolf M. J. P.: The Video Game Explosion: A History from PONG to Playstation and Beyond, Greenwood Press, 2008. ISBN: 9780313338687

[12] Boot W. R., Kramer A. F., Simons D. J., Fabiani M., Gratton G.: The effects of video game playing on attention, memory, and executive control, Acta Psychologica, 2008. DOI: 10.1016/j.actpsy.2008.09.005

[13] Madigan J.: Getting Gamers: The Psychology of Video Games and Their Impact on the People Who Play Them, Rowman & Littlefield, 2015. ISBN: 9781442239999

[14] Ryan R. M., Deci E. L.: Self-determination theory and the facilitation of intrinsic

motivation, social development, and well-being, American Psychologist, 2000. DOI: 10.1037/0003-066X.55.1.68

[15] Harvey A.: Twisty Little Passages: An Approach to Interactive Fiction, MIT Press, 2014. ISBN: 9780262134361

[16] Jerald J.: The VR Book: Human-Centered Design for Virtual Reality, Morgan & Claypool, 2015. ISBN: 9781970001150

[17] Togelius J., Yannakakis G. N., Stanley K. O., Browne C.: Search-based procedural content generation: A taxonomy and survey, IEEE Transactions on Computational Intelligence and AI in Games, 2011. DOI: 10.1109/TCIAIG.2011.2148116

[18] Yuan B., Folmer E., Harris F. C.: Game accessibility: A survey, Universal Access in the Information Society, 2011. DOI: 10.1007/s10209-010-0189-5

[19] Zadeh L. A.: Fuzzy sets, Information and Control, vol. 8, no. 3, 1965, pp. 338–353. DOI: 10.1016/S0019-9958(65)90241-X

[20] Ross T. J.: Fuzzy Logic with Engineering Applications, 3rd edition, John Wiley & Sons, 2010. ISBN: 9780470743768

Mellékletek

```
private void Update()
{
    if (Input.GetKeyDown(KeyCode.UpArrow) || Input.GetKeyDown(KeyCode.W))
    DoJump();

    if ((Input.GetKeyUp(KeyCode.UpArrow) || Input.GetKeyUp(KeyCode.W)) &&
    body.velocity.y > 0)
    body.velocity = new Vector2(body.velocity.x, body.velocity.y * 0.5f);
}

private void DoJump()
{
    if (coyoteTimer < 0 && !CheckWall() && jumpsleft <= 0)
    return;

    if (SoundManager.instance != null)
    SoundManager.instance.PlaySound(jumpSound);

    if (CheckWall() && !CheckGround())
    ExecuteWallJump();
    else
    {
    if (CheckGround() || coyoteTimer > 0)
        body.velocity = new Vector2(body.velocity.x, jumpStrength);
    else
    {
        if (coyoteTimer > 0)
            body.velocity = new Vector2(body.velocity.x, jumpStrength);
         else

            if (jumpsleft > 0)
            {
                body.velocity = new Vector2(body.velocity.x, jumpStrength);
                jumpsleft--;
            }
        }
    }
    coyoteTimer = 0;
    }
}
```

2. kódrészlet: Player Ugrás logika részlet

```
public void TakeDamage(float damage)
{
    if (currentHealth <= 0)
        Die();
    else
    {
        anim.SetTrigger("hurt");
        StartHurtInvulnerability();
        if (hurtClip != null)
            SoundManager.instance.PlaySound(hurtClip);
    }
}
private void Die()
{
    if (isDead) return;
    isDead = true;

    if (CompareTag("Player"))
    {
        //Nyissuk ki az összes szobát halálkor (inaktívakat is megtaláljuk)
        foreach (var roomActivator in FindObjectsOfType<ActivateRoomPoint>(true))
        {
            if (roomActivator != null && roomActivator.objectToShow != null)
            {
                roomActivator.objectToShow.SetActive(false);
            }
        }
        deathCount++;
        isGameOver = true;
        if (uIManager != null)
        {
            //Tiltsuk le a pause-t halál után
            uIManager.DisablePause();
            uIManager.StartCoroutine(uIManager.ShowGameOverScreenWithDelay(1f));
        }

public void Respawn()
{
    isDead = false;
    currentHealth = startingHealth;

    if (anim != null)
    {
        anim.ResetTrigger("die");
        anim.SetBool("dead", false);
        anim.Play("Idle", 0, 0f);
    }

    if (rb != null)
```

```
        {
            rb.constraints = originalConstraints;
            rb.gravityScale = originalGravityScale;
            rb.velocity = Vector2.zero;
        }

        if (col != null)
             col.enabled = true;

        foreach (var comp in components)
            comp.enabled = true;

        StartHurtInvulnerability();

        //Pause újra engedélyezése
        if (uIManager != null)
            uIManager.EnablePause();
        foreach (var trigger in FindObjectsOfType<BossRoomTrigger>())
        {
            trigger.ResetTriggerState();
        }
}
```

4. kódrészlet: Player TakeDamage, Die és Respawn metódusainak részletei

```
private void Update()
{
    if (PlayerInSight())
    {
        if (cooldownTimer >= attackCooldown && playerHealth != null &&
                                playerHealth.currentHealth > 0)
        {
            cooldownTimer = 0f;
            anim.SetTrigger("meleeAttack");
            SoundManager.instance.PlaySound(attackSfx);
        }
    }
}

private bool PlayerInSight()
{
    RaycastHit2D hit = Physics2D.BoxCast(
        boxCollider.bounds.center + transform.right * attackRange *
        transform.localScale.x * colliderDistance,
        new Vector3(boxCollider.bounds.size.x * attackRange, boxCollider.bounds.size.y,
        boxCollider.bounds.size.z),
        0f,
        Vector2.left,
```

```
            0f,
            playerLayer
        );

        if (hit.collider != null)
        {
            playerHealth = hit.collider.GetComponent<Health>();
            return playerHealth != null;
        }

        playerHealth = null;
        return false;
}
```

5. kódrészlet: MeleeEnemy Update és PlayerInSight metódusa

```
public void ResetEnemy()
{
    cooldownTimer = Mathf.Infinity;

    if (selfHealth != null)
        selfHealth.ResetHealth();
a
    if (enemyPatrol != null)
        enemyPatrol.enabled = true;

    if (anim != null)
    {
        anim.Rebind();
        anim.Update(0f);
    }
}
```

8. kódrészlet: ResetEnemy metódus

```
private void ScanForPlayer()
{
    UpdateDirections();

    //check if spikehead sees player in all 4 directions
    for (int i = 0; i < possibleDirections.Length; i++)
    {
        Debug.DrawRay(transform.position, possibleDirections[i], Color.red);
        RaycastHit2D hit = Physics2D.Raycast(transform.position, possibleDirections[i],
                    detectionRange, playerLayer);

        if (hit.collider != null && !isAttacking)
        {
            isAttacking = true;
            moveDirection = possibleDirections[i];
            checkTimer = 0f;
        }
    }
}
```

9. kódrészlet: SpikeHead ScanForPlayer metódus

```
private void ShootArrow()
{
    cooldownTimer = 0f;

    SoundManager.instance.PlaySound(arrowSfx);
    GameObject arrow = arrows[FindInactiveArrow()];
    arrow.transform.position = spawnPoint.position;
    arrow.GetComponent<EnemyProjectile>().ActivateProjectile();
}

private int FindInactiveArrow()
{
    for (int i = 0; i < arrows.Length; i++)
    {
        if (!arrows[i].activeInHierarchy)
            return i;
    }
    return 0;
}
```

10. kódrészlet: ShootArrow és FindInactiveArrow metódusok

```
private void OnCollisionEnter2D(Collision2D collision)
{
    if (!isFalling && collision.gameObject.CompareTag("Player"))
    {
        isFalling = true;
        Invoke(nameof(StartFalling), fallDelay);
    }
}

private void StartFalling()
{
    rb.bodyType = RigidbodyType2D.Dynamic; // mostantól a fizika irányítja
    Destroy(gameObject, destroyDelay);  // pár mp múlva eltűnik
}
```

11. kódrészlet:OnCollisionEnter2D és StartFalling metódusok

```
private void Update()
{
    if (bossHealth == null || healthBarFill == null)
        return;

    float fillAmount = (float)bossHealth.currentHealth / bossHealth.startingHealth;
    healthBarFill.fillAmount = Mathf.Clamp01(fillAmount);

    if (healthPercentageText != null)
    {
        int percent = Mathf.RoundToInt(fillAmount * 100);
        healthPercentageText.text = percent + "%";
    }
}
```

12. kódrészlet: Update metódus Boss HealthBar UI frissítésére

```
public void StartPowerupInvulnerability(float duration)
{
    if (hurtCoroutine != null)
    {
        StopCoroutine(hurtCoroutine);
        hurtCoroutine = null;
    }

    if (powerupCoroutine != null) StopCoroutine(powerupCoroutine);
    powerupCoroutine = StartCoroutine(InvulnerabilityPowerupCoroutine(duration));
}

private IEnumerator InvulnerabilityPowerupCoroutine(float duration)
{
```

```
    isInvulnerable = true;
    spriteRenderer.color = new Color(1f, 1f, 1f, 0.5f);
    yield return new WaitForSeconds(duration);
    spriteRenderer.color = Color.white;
    isInvulnerable = false;
    powerupCoroutine = null;
}
```

14. kódrészlet: StartPowerupInvulnerability és InvulnerabilityPowerupCoroutine metódusok

```
    public void ApplySpeedBoost(float duration, float speedMultiplier, float jumpMultiplier, int extraJumps)
{
    if (speedBoostCoroutine != null)
        StopCoroutine(speedBoostCoroutine);

    speedBoostCoroutine = StartCoroutine(SpeedBoostCoroutine(duration,
                                speedMultiplier, jumpMultiplier, extraJumps));
}
```

15. kódrészlet: ApplySpeedBosst metódus

```
private BossTactic DecideTactic()
{
	float hp = memory.BossHealthPercent();
	float dist = memory.PlayerDistanceFraction(transform, player);
	float aggr = memory.AggressionLevel;
	float jump = memory.JumpTendency;

	float hpLow = FuzzyLogic.Low(hp);
	float hpHigh = FuzzyLogic.High(hp);
	float distNear = FuzzyLogic.Low(dist);
	float distFar = FuzzyLogic.High(dist);
	float aggrHigh = FuzzyLogic.High(aggr);
	float aggrLow = FuzzyLogic.Low(aggr);
	float jumpHigh = FuzzyLogic.High(jump);

	float evadeRule = FuzzyLogic.And(hpLow, distNear);
	float defensiveRule = FuzzyLogic.And(distNear, aggrLow);
	float rangedRule = FuzzyLogic.Or(distFar, jumpHigh);
	float aggressiveRule = FuzzyLogic.And(aggrHigh, FuzzyLogic.Not(hpLow));

	float max = Mathf.Max(aggressiveRule, defensiveRule, rangedRule, evadeRule);
	BossTactic chosen = BossTactic.Evade;
	if (max == aggressiveRule) chosen = BossTactic.Aggressive;
	else if (max == defensiveRule) chosen = BossTactic.Defensive;
	else if (max == rangedRule) chosen = BossTactic.Ranged;

	return chosen;
}
```

18. kódrészlet: DecideTactic metódus

```
private float CalculateFuzzyScore(BossAttackBase attack, BossTactic tactic)
{
    float dist = memory.PlayerDistanceFraction(transform, player);
    float aggr = memory.AggressionLevel;
    float jump = memory.JumpTendency;
    float hp = memory.BossHealthPercent();
    float playerHeight = memory.PlayerHeightFraction(transform, player);

    float distNear = FuzzyLogic.Low(dist);
    float distFar = FuzzyLogic.High(dist);
    float aggrHigh = FuzzyLogic.High(aggr);
    float jumpHigh = FuzzyLogic.High(jump);
    float hpLow = FuzzyLogic.Low(hp);

    float baseScore = 0f;

    if (attack is BossFireballAttack)
        baseScore = FuzzyLogic.SoftOr(distFar, jumpHigh) * 10f;
    else if (attack is BossDashAttack)
        baseScore = FuzzyLogic.SoftAnd(distNear, aggrHigh) * 10f;
    else if (attack is BossAttackUp)
        baseScore = FuzzyLogic.SoftAnd(jumpHigh, aggrHigh) * 8f +
        FuzzyLogic.High(playerHeight) * 5f;
    else if (attack is BossAttackDown)
        baseScore = FuzzyLogic.SoftAnd(distNear, hpLow) * 7f +
        FuzzyLogic.Low(playerHeight) * 5f;

    switch (tactic)
    {
        case BossTactic.Aggressive: baseScore *= 1.2f; break;
        case BossTactic.Defensive: baseScore *= 0.9f; break;
        case BossTactic.Ranged: baseScore *= 1.1f; break;
        case BossTactic.Evade: baseScore *= 0.6f; break;
    }

    float learned = memory.GetEffectiveness(attack);
    baseScore = Mathf.Lerp(baseScore, baseScore * 1.5f, learned);
    baseScore += UnityEngine.Random.Range(-1f, 1.5f);
    return Mathf.Clamp(baseScore, 0f, 15f);
}
```

19. kódrészlet: CalculateFuzzyScore

```
void Update()
{
    ObservePlayer();
    TrackDamageDuringMove();
    DecayMemory();
}
```

20. kódrészlet: BossLearningMemory Update metódusa

```
public void ObservePlayer()
{
    if (player == null) return;

    Vector2 delta = (Vector2)player.position - lastPlayerPos;

    // Jump tendency figyelés
    if (delta.y > 0.1f)
        jumpTendency = Mathf.Clamp01(jumpTendency + 0.05f);
    else
        jumpTendency = Mathf.Clamp01(jumpTendency - 0.02f);

    // Aggresszió a player távolsága alapján
    float dist = Vector2.Distance(player.position, transform.position);
    float proximity = Mathf.Clamp01(1f - dist / 5f);
    adaptiveAggressionMemory = Mathf.Lerp(adaptiveAggressionMemory, proximity,
    0.1f);

    aggressionLevel = Mathf.Lerp(aggressionLevel, adaptiveAggressionMemory, 0.05f);

    lastPlayerPos = player.position;
}
```

21. kódrészlet: ObservePlayer metódus

```
private void DecayMemory()
{
    for (int i = 0; i < attackStats.Count; i++)
        attackStats[i].lastSuccess = Mathf.Lerp(attackStats[i].lastSuccess, 0.5f, 1f -
        decayRate);

    for (int i = 0; i < moveStats.Count; i++)
        moveStats[i].lastSuccess = Mathf.Lerp(moveStats[i].lastSuccess, 0.5f, 1f -
        decayRate);

    aggressionLevel = Mathf.Lerp(aggressionLevel, 0.5f, 1f - decayRate);
    jumpTendency = Mathf.Lerp(jumpTendency, 0.5f, 1f - decayRate);
}
```

22. kódrészlet: DecayMemory metódus

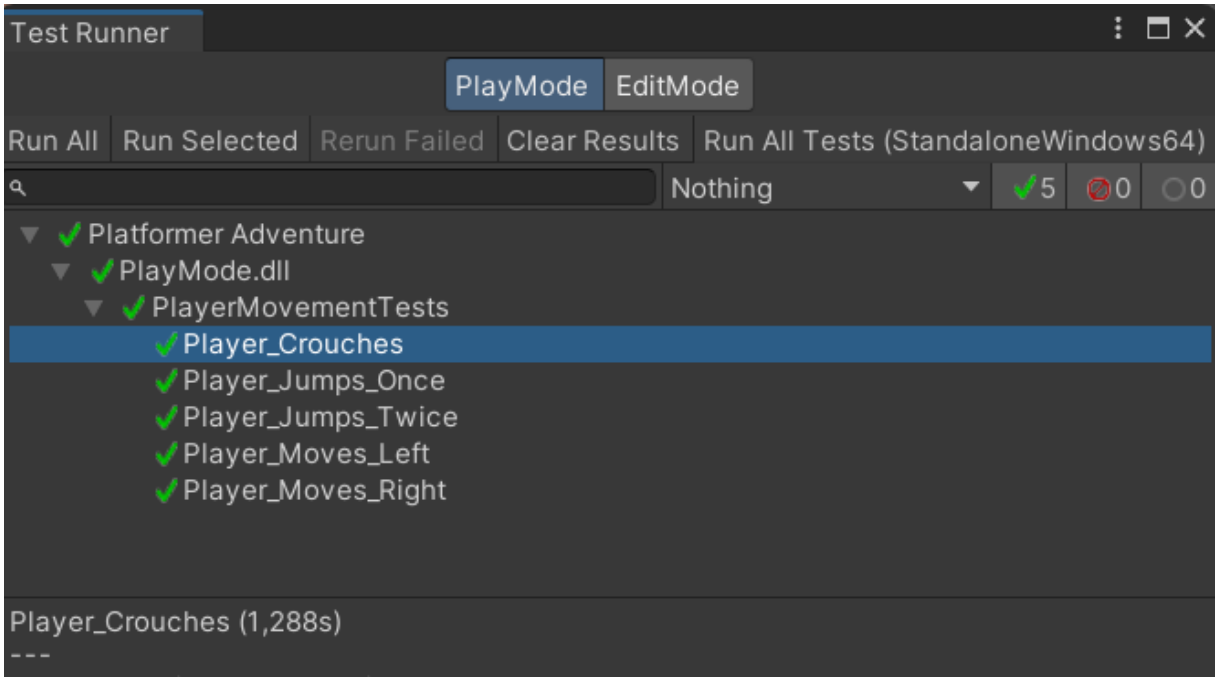
```
private bool ShouldTeleportHigh()
{
    if (memory == null || bossHealth == null) return false;

    float hp = memory.BossHealthPercent();
    float aggr = memory.AggressionLevel;
    float jump = memory.JumpTendency;
    float dmg = memory.HealthSinceLastMove;

    float hpLow = FuzzyLogic.Low(hp);
    float aggrHigh = FuzzyLogic.High(aggr);
    float jumpHigh = FuzzyLogic.High(jump);
    float dmgHigh = FuzzyLogic.High(dmg / 5f);

    // Soft fuzzy szabály
    float rule = FuzzyLogic.SoftAnd(FuzzyLogic.SoftOr(hpLow, dmgHigh),
                 FuzzyLogic.SoftOr(aggrHigh, jumpHigh));

    // Random faktor hozzáadva
    rule += Random.Range(-0.1f, 0.2f);
    rule = Mathf.Clamp01(rule);

    return Random.value < rule;
}
```

23. kódrészlet: ShouldTeleportHigh metódus

```
private IEnumerator HealthPickupRespawnRoutine(GameObject pickup)
{
    while (bossFightActive)
    {
        if (pickup != null && !pickup.activeSelf)
        {
            float respawnDelay = GetAdaptiveRespawnDelay();
                yield return new WaitForSeconds(respawnDelay);
                pickup.SetActive(true);
        }
         else
        {
                yield return null;
        }
    }
}
private float GetAdaptiveRespawnDelay()
{
    if (bossMemory == null || bossHealth == null || playerHealth == null)
        return Random.Range(minRespawnDelay, maxRespawnDelay);
```

```
    float healthPercent = bossMemory.BossHealthPercent();
    float aggression = bossMemory.AggressionLevel;
    float jumpTendency = bossMemory.JumpTendency;
    float deathsFactor = Mathf.Clamp01((float)deathsDuringBossFight / 5f);

    // --- Fuzzy szabályok ---
    float healthLow = FuzzyLogic.Low(healthPercent);
    float aggrHigh = FuzzyLogic.High(aggression);
    float jumpHigh = FuzzyLogic.High(jumpTendency);
    float deathsHigh = FuzzyLogic.High(deathsFactor);

    float fuzzyScore = FuzzyLogic.Or(FuzzyLogic.Or(healthLow, aggrHigh),
                                     FuzzyLogic.Or(jumpHigh, deathsHigh));
    fuzzyScore = Mathf.Clamp01(fuzzyScore);

    float respawnDelay = Mathf.Lerp(maxRespawnDelay, minRespawnDelay, fuzzyScore);
    respawnDelay *= Random.Range(0.9f, 1.1f);

    return respawnDelay;
}
```

24. kódrészlet: HealthPickupRespawnRoutine és GetAdaptiveRespwanDelay metódusok

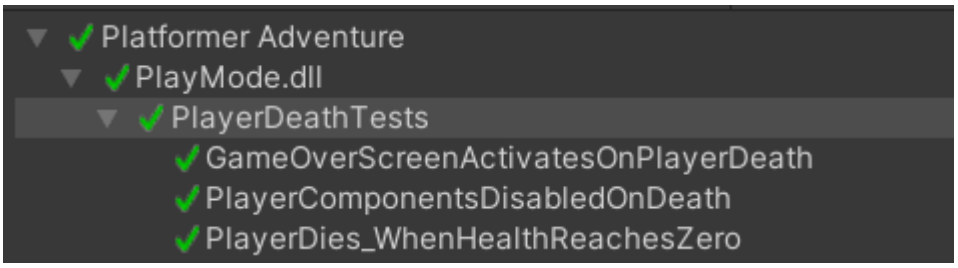


38. ábra: PlayerMovement tesztek

```
public void SetHorizontalInput(float value)
{
    useTestingInput = true;
    testingHorizontalInput = value;
```

```
}
[UnityTest]
public IEnumerator Player_Moves_Left()
{
    float startX = player.transform.position.x;

    movement.SetHorizontalInput(-1f); // balra

    yield return new WaitForSeconds(0.2f);

    Assert.Less(player.transform.position.x, startX, "A játékosnak balra kellene
    mozognia!");
}
```

25. kódrészlet: SetHorizontalInput és Player_Moves_Left metódusok

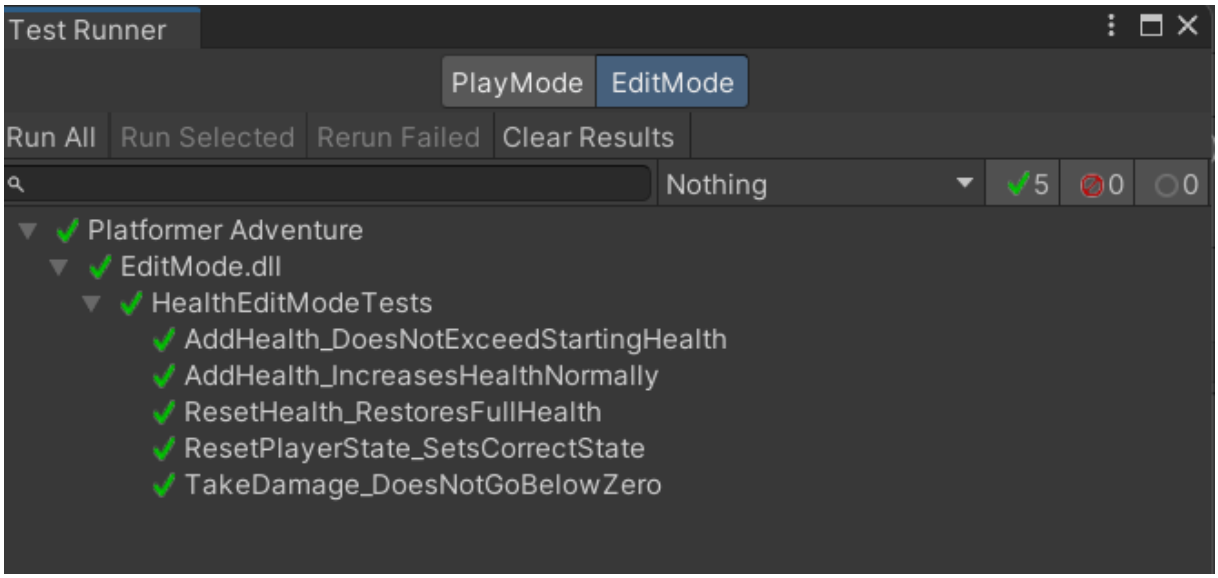


39. ábra: PlayerHealth Play Mode tesztek

```
[UnityTest]
public IEnumerator Trigger_ActivatesBossAndRestoresPlayerHealth()
{
    // Player élete legyen alacsony, hogy lássuk a visszatöltést
    playerHealth.currentHealth = 1f;

    // Előzőleg biztosítsuk, hogy boss inaktív felébredés szempontjából
    boss.ResetBoss();

    // Player belép a triggerbe
    BoxCollider2D playerCollider = player.GetComponent<BoxCollider2D>();
    if (playerCollider == null)
        player.AddComponent<BoxCollider2D>();

    // Fizikailag aktiváljuk a trigger-t (OnTriggerEnter meghívása)
    triggerObject.GetComponent<Collider2D>().isTrigger = true;
    triggerObject.GetComponent<BossRoomTrigger>().ResetTriggerState();
    triggerObject.GetComponent<BossRoomTrigger>().SendMessage("OnTriggerEnter2D
    ", playerCollider, SendMessageOptions.DontRequireReceiver);

    yield return null; // 1 frame, hogy a trigger logika lefusson
```

```
    // Ellenőrizzük, hogy a boss felébredt
    Assert.IsTrue(boss.BossIsAwake(), "A bossnak fel kellett volna ébrednie a trigger hatására!");

    // Ellenőrizzük, hogy a player élete visszatöltődött
    Assert.AreEqual(playerHealth.startingHealth, playerHealth.currentHealth, "A player élete nem lett visszatöltve!");
}
```

26. kódrészlet: Trigger_ActivatesBossAndRestoresPlayerHealth metódus



40. ábra: PlayerHealth Edit Mode tesztek

```
//1. Ha van inaktív projectile → azt adja vissza
[Test]
public void FindInactiveProjectile_ReturnsCorrectIndex()
{
    // Arrange
    projectiles[0].SetActive(true);
    projectiles[1].SetActive(false);
    projectiles[2].SetActive(true);

    // Private metódus meghívása reflectionnel
    int index = (int)rangedEnemy.GetType()
        GetMethod("FindInactiveProjectile", System.Reflection.BindingFlags.NonPublic |
        System.Reflection.BindingFlags.Instance).Invoke(rangedEnemy, null);

    // Assert
    Assert.AreEqual(1, index);
}

//2. Ha minden projectile aktív → fallback = 0
```

```
[Test]
public void FindInactiveProjectile_WhenAllActive_ReturnsZero()
{
    // Arrange
    projectiles[0].SetActive(true);
    projectiles[1].SetActive(true);
    projectiles[2].SetActive(true);

    int index = (int)rangedEnemy.GetType().GetMethod("FindInactiveProjectile",
                    System.Reflection.BindingFlags.NonPublic |
                    System.Reflection.BindingFlags.Instance).Invoke(rangedEnemy,
                    null);

    Assert.AreEqual(0, index);
}
```

27. kódrészlet: FindInactiveProjectile_ReturnsCorrectIndex és FindInactiveProjectile_WhenAllActive_ReturnsZero metódusok

41. ábra: DeathScreen

A játék fejlesztése során létrehozott forráskódok a következő GitHub linken érhetőek el:

https://github.com/AttilaBalogh14/Szakdolgozat.git